



Pitch Visual OMT™ User's Guide

«The Premier HLA Object Model Editor»

Version 2.7

Table of Contents

1. Introduction	3
1.1. About this document	3
1.2. About Pitch	3
1.3. Licensing	4
2. Concept of Operations	5
2.1. About FOMs and Modular FOMs	5
2.2. A graphical tool	6
2.3. A project-based tool	6
2.4. An intelligent tool	7
2.5. HLA 4 Preview Features	7
2.6. Further Reading	8
3. Installing Pitch Visual OMT™	9
3.1. Installation	9
3.2. Activating the license	13
4. An Overview of Pitch Visual OMT™	14
4.1. Opening a sample project	14
4.2. Overview of the GUI	14
4.3. Navigating Modules and Tables	15
4.4. The Module Overview	15
4.5. Understanding the Module Class Tree	16
4.6. Datatypes	17
4.7. A Look at the Issue List	17
4.8. Searching Elements and Inspecting References	18
4.9. Convenience Features	18
4.10. Import and Export	18
4.11. Tools	19
5. Setting up a Project	20
5.1. Before you Start	20
5.2. Understanding Allowed Dependencies	20
5.3. Creating a Project	20
5.4. Adding a module	21
5.5. The Overview and the Identification table	23
5.6. Other Project settings	27
6. Working with Class Trees	29
6.1. Drag and drop within a module	29

6.2. Drag and drop between modules	30
6.3. Object Classes and Attributes	30
6.4. Interaction Classes and Parameters	32
6.5. How classes are displayed	32
6.6. Scaffolding and Dependencies	33
7. Working with Datatypes	35
7.1. Datatype overview	35
7.2. Basic Datatypes	36
7.3. Simple Datatypes	36
7.4. Enumerated Datatypes	37
7.5. Array Datatypes	38
7.6. Fixed Record Datatypes	38
7.7. Variant Record Datatype	39
8. Working with Other Tables	41
8.1. Synchronization Points	41
8.2. Time Representation	41
8.3. Dimensions	42
8.4. Transportation Types	43
8.5. User Supplied Tags	43
8.6. Update Rates	44
8.7. Switches	44
8.8. Notes	44
8.9. Service Utilization	45
9. Advanced Topics	46
9.1. Refactoring	46
9.2. Tips for splitting a FOM into modules	46
9.3. Verifying RTI compliance	47
9.4. Export to PDF	47
9.5. Export to RTF	47
9.6. Importing and exporting the HLA 1516-2000 format	48
9.7. Importing and exporting the HLA 1.3 OMT format	48
9.8. Importing and exporting the HLA 1.3 FED format	49
9.9. Saving a Merged HLA 1516-2010 FOM	49
9.10. Importing and exporting the HLA 4 FOM format	50
9.11. Comparing Modules	50
9.12. Uniform DDM Tool	52
9.13. HLA 4 Extended Merge	55

1. Introduction

1.1. About this document

This document describes how to install, configure and use Pitch Visual OMT™, the premier editor for HLA Object Models. Pitch Visual OMT™ enables you to work with both older and newer versions of HLA.

Readers of this document are assumed to have some familiarity with HLA simulations.

1.2. About Pitch

Pitch is a leading supplier of commercial, standards-based infrastructure tools and services to the modeling and simulation community.

Well-known tools from Pitch include:

Pitch Booster™

Connect simulations across WAN with performance and fault tolerance.

Pitch CDS Gateway™

Connect federations of different security classifications.

Pitch Commander™

Monitor and control your simulations across your organization.

Pitch Debrief™

Add synchronized debrief capabilities to simulation and training exercises

Pitch Developer Studio™

Generate a royalty-free HLA and DIS module in C++, C# or Java for your simulation.

Pitch DIS Adapter™

Integrate your DIS applications and your HLA federations with monitoring and filtering.

Pitch Extender™

Connect federations with different RTIs, FOMs or different performance profiles.

Pitch KML Adapter™

Visualize your scenario in the KML viewers such as Google Earth.

Pitch Media Streamer™

Integrate video and simulation data in your simulation infrastructure.

Pitch pRTI™

Make your systems work together using the open international HLA standard.

Pitch Recorder™

Record, inspect, play back and export your HLA, DIS, Link 16 and other simulation data.

Pitch Talk™

Communicate using voice, intercom, radio and chat with scalability and management.

Pitch Unreal Engine Connector™

Connect your Simulation to the Unreal Engine using HLA.

Pitch Visual OMT™

Create, maintain, visualize and analyze object models for information exchange.

Read more about Pitch Technologies at www.pitchtechnologies.com.

1.3. Licensing

Pitch Visual OMT™ is licensed per host.

2. Concept of Operations

Pitch Visual OMT™ is the leading tool for developing and maintaining object models for the HLA standard. It enables you to develop using the latest standard, while at the same time supporting federations using both older and newer versions of the standard.

2.1. About FOMs and Modular FOMs

The *Federation Object Model* (FOM) is a data model that describes the information exchange taking place when several systems execute together in an HLA federation. It specifies what shared objects (e.g., aircraft) exist, what attributes (e.g., altitude and speed) they have, as well as the datatypes for all attributes. It also describes exchanged interactions, like commands, radio messages and explosions. A FOM is a major component in the Federation Agreement Document (FAD), which should also include a document describing the overarching design of the federation.

The HLA 1516-2010 standard introduces *modular* FOMs. These can be used to separate different aspects of the information exchange. For example, it is possible to have one FOM module describing how information about aircraft and ground vehicles is exchanged, another module describing radio messages, and a third module describing how the federation is started and stopped. [Figure 1](#) shows how FOM modules can be developed using Visual OMT and then used in a federation.

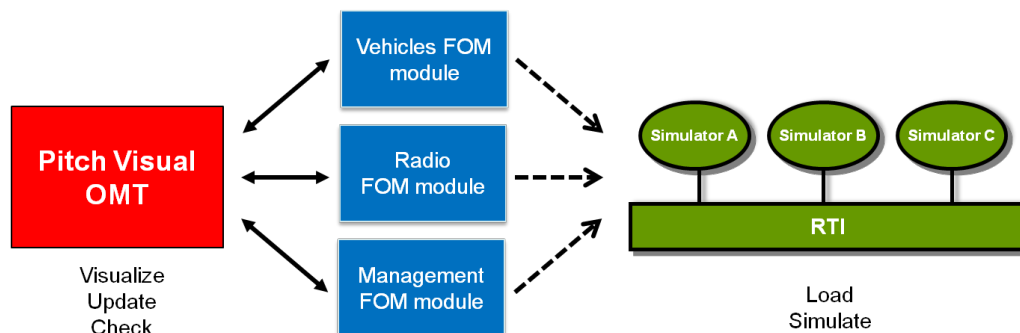


Figure 1. Developing and using FOM Modules.

Visual OMT enables you to build projects consisting of several FOM modules. It enables you to visualize and edit single FOM modules or combinations of FOM modules. It also enables you to find and resolve any inconsistencies between FOM modules.

Visual OMT also enables you to develop *Simulation Object Models* (SOMs), which describe what information a specific federate (simulation) can publish and subscribe in a potential federation. A SOM can be seen as a specification for the capabilities of a particular simulator, with respect to interoperability.

2.2. A graphical tool

Visual OMT provides a graphical user interface that visualizes the combination of all information in the FOM modules, such as objects, interactions, datatypes, etc.

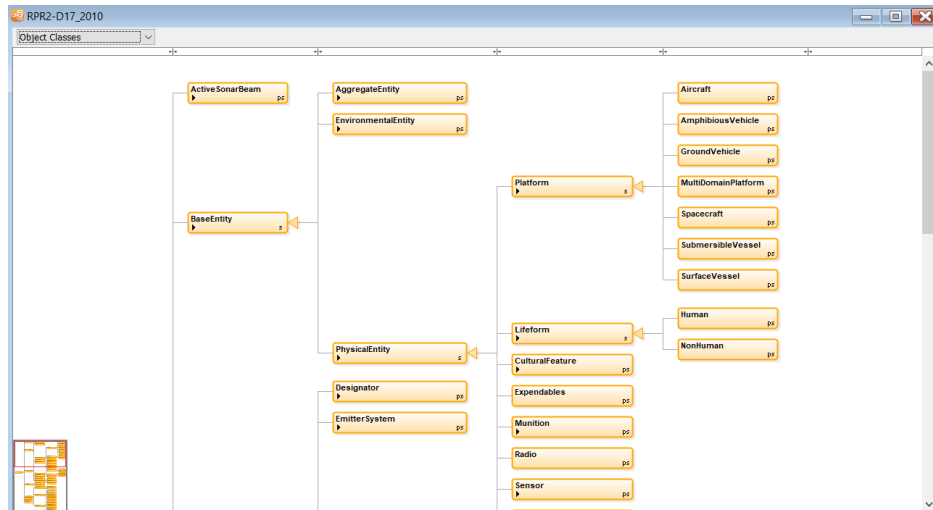


Figure 2. The object class structure of the RPR FOM, including a mini map.

Class trees (like the one in [Figure 2](#)) are used to visualize the class hierarchies. Several FOM modules can be overlaid to show the resulting combined class tree. There is also an overview of the entire project, enabling you to explore exactly which FOM modules exist and of which tables they consist.

There are also tabular overviews that can be sorted according to different columns and in some cases filtered. Classes, datatypes and other elements of the FOM can be edited using drag and drop between modules. There are also **[Undo]** and **[Redo]** buttons to facilitate experimentation and undo any mistakes in editing.

2.3. A project-based tool

Visual OMT is centered on projects that contain several FOM modules. There are typically three types of FOM modules in a project:

- Regular modules: modules that you develop.
- Read-only modules: for example, the predefined HLStandardMIM module or reference FOMs, like the RPR FOM. These should be marked as "Read-only" in the project, so that you don't edit them by mistake. If someone else is responsible for maintaining a certain module in your project, you may also mark it as "Read-only".
- Temporary modules: modules that you open for reference or for copying some useful elements. These should be marked as "Exclude Module from error checking".

This is illustrated in [Figure 3](#):

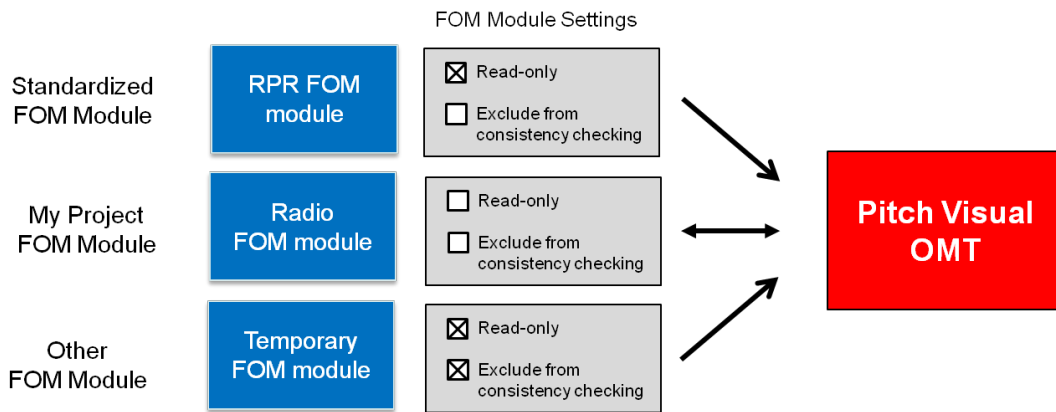


Figure 3. Three types of modules.

FOM modules may depend on other FOM modules. A specialized aircraft module may, for example, extend the aircraft class of the standardized RPR FOM. It is important that you don't introduce any unwanted dependencies between modules. Visual OMT enables you to specify exactly what dependencies are allowed within your project: for example, you might specify that a specialized aircraft module can use elements from the general RPR FOM but not the other way around.

2.4. An intelligent tool

The user interface of Visual OMT provides useful tips for developing good FOM modules. There is also a rule engine that continuously checks all modules for errors and conflicts, and presents them in an issue list (see Figure 4). It gives you hints about best practices for FOM development, and lets you fix some of the most common mistakes with a single click. You can read more about this in Section 4.7.

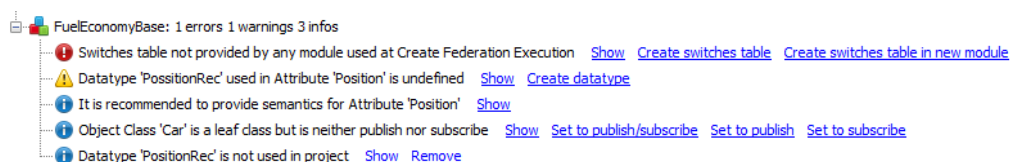


Figure 4. Sample Issue List.

Visual OMT also keeps track of the usage of all elements: you can use the intelligent search function to find definitions of elements, and to quickly locate where they are being used.

2.5. HLA 4 Preview Features

Visual OMT includes HLA 4 Preview features, giving you the opportunity to experiment with some of the new features being developed for the upcoming version of the HLA standard. As HLA 4 is not yet released, the HLA 4 preview features are subject to change.

The following HLA 4 Preview features are available:

- **Extended Merge** that makes it possible to add attributes and parameters to existing

classes, extend enumerators and variant records, and add dimensions to attributes and parameters when merging FOM modules. See [Section 9.13](#) for more information.

2.6. Further Reading

For a deeper understanding of FOMs and FOM development, see the following recommended reading:

"Processes and Tools for Management and Reuse of FOM Modules" (SISO paper 10F-SIW-031), which gives an overview of the life cycle of FOM modules as well as the philosophy behind Visual OMT. This paper can be downloaded from www.pitchtechnologies.com.

"Getting started with FOM Modules" (SISO paper 09F-SIW-082), which is a thorough technical introduction to FOM modules, including principles for combining modules and some best-practice advice. This paper can be downloaded from www.pitchtechnologies.com.

"IEEE 1516.2-2010 High Level Architecture (HLA) - Object Model Template (OMT) Specification", which is the full and authoritative standard of the HLA Evolved FOM. Available from shop.ieee.org.

3. Installing Pitch Visual OMT™

This chapter covers how to install Visual OMT on Windows. Before you start the installation, check that you have:

- The installation executable.
- The license activation key.

3.1. Installation

Start the installation by simply double-clicking on the installer file. A graphical installer will now start.

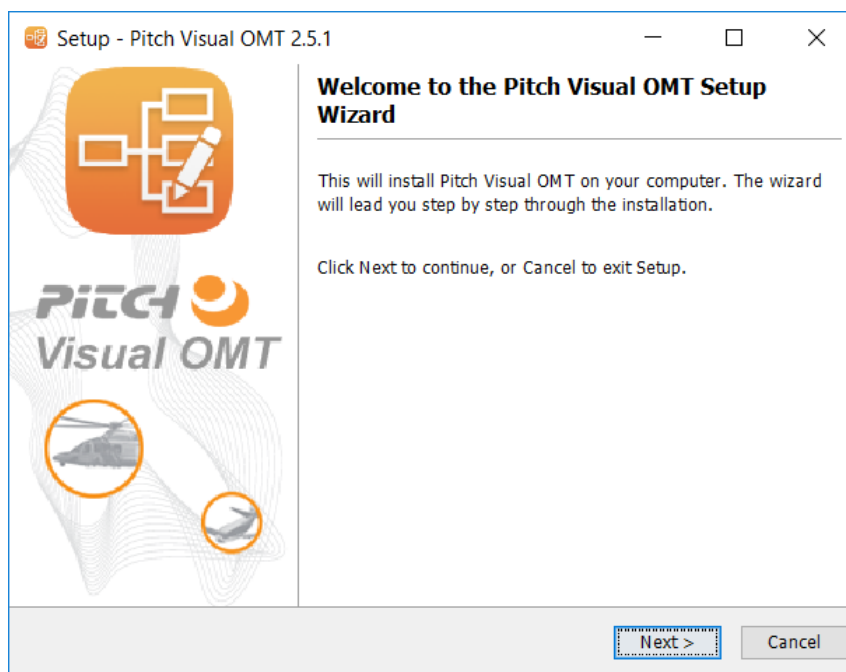


Figure 5. Installation introduction.

[Figure 5](#) shows the first screen that is displayed. Click [**Next**] to continue. The license agreement shown in [Figure 6](#) will be displayed.

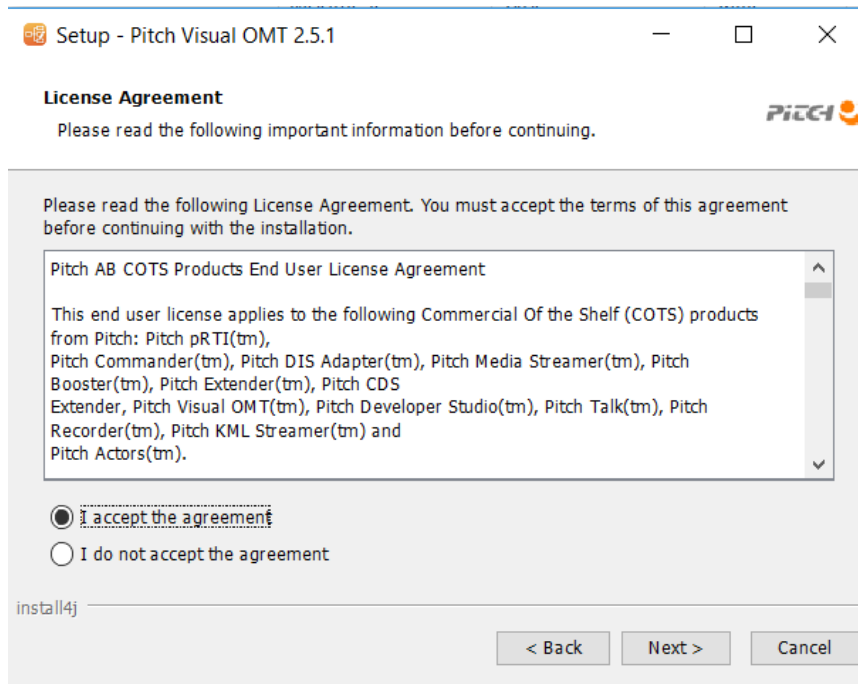


Figure 6. End-user license agreement.

Read the license agreement and make the appropriate selection. Click **[Next]** to continue. The screen shown in [Figure 7](#) allows you to specify the installation folder. The default folder is C:\ProgramFiles\PitchVisualOMT2. Click **[Next]** to continue.

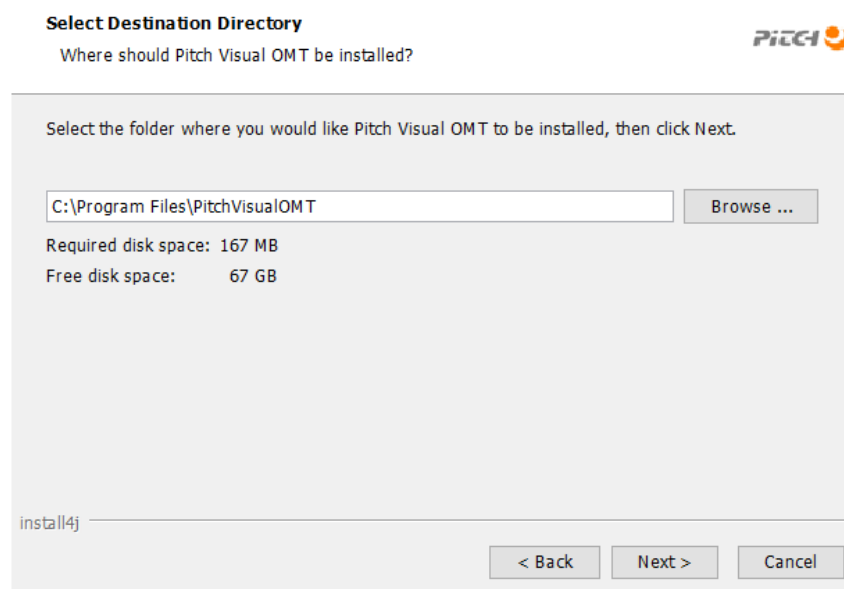


Figure 7. Choose install folder.

The screen shown in [Figure 8](#) allows you to specify where to place the shortcut folder for Visual OMT. By default, it is placed in the Visual OMT program group. Click **[Next]** to continue.

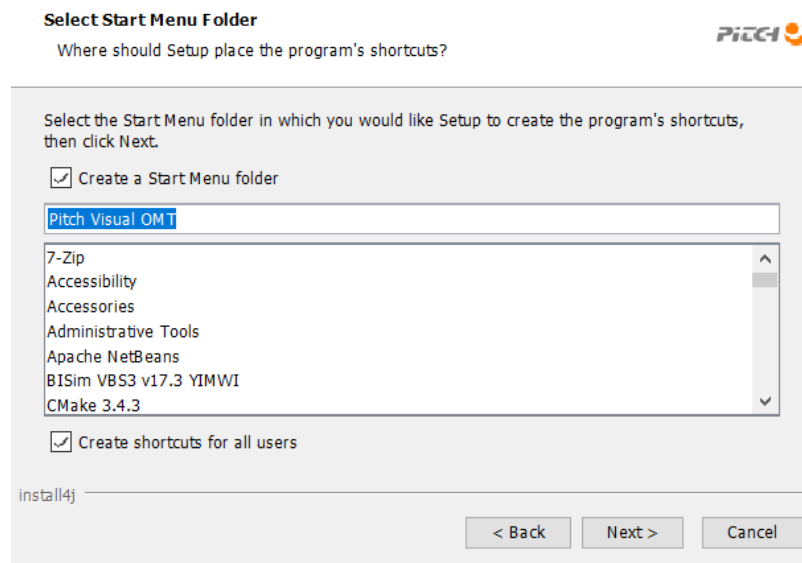


Figure 8. Create icons in the start menu.

The screen shown in [Figure 9](#) allows you to associate files with the file extension `.vomt` (the Visual OMT project format) with the application. Click **[Next]** to continue.

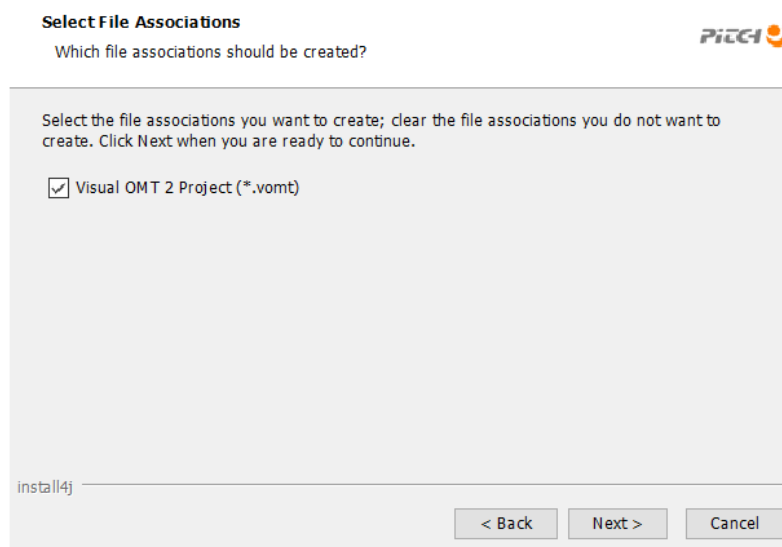


Figure 9. Create file associations.

In the screen shown in [Figure 10](#), you can select to create a desktop icon.

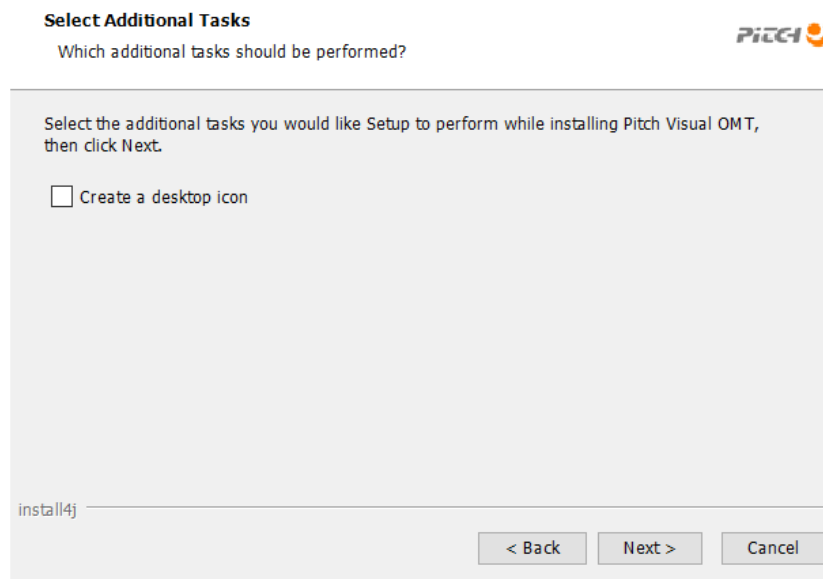


Figure 10. Select additional tasks.

Click [**Next**] to complete the installation.

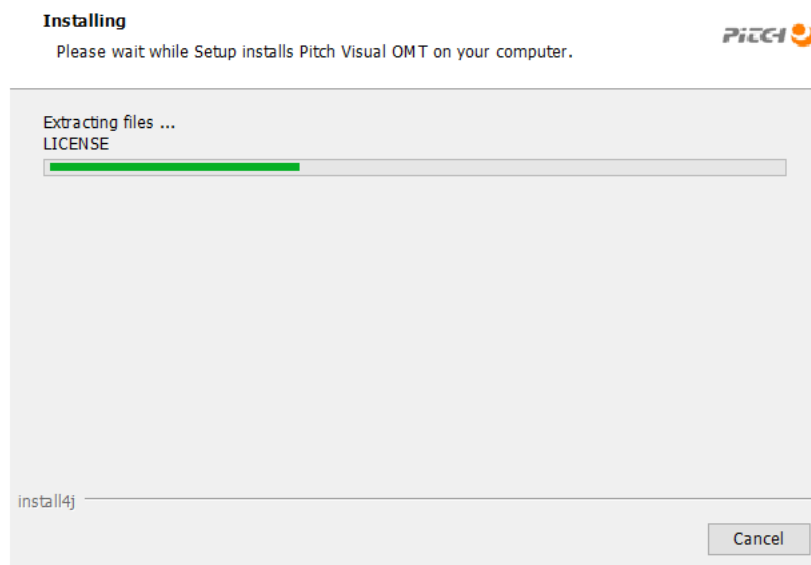


Figure 11. Installation process.

When the installation is complete, you will see the screen shown in [Figure 12](#).

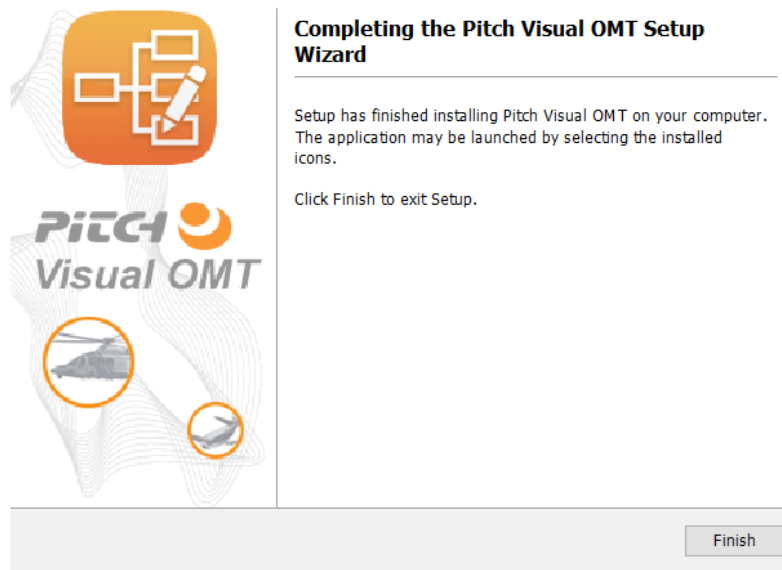


Figure 12. Installation complete.

You are now ready to start using Visual OMT.

3.2. Activating the license

The first time you start the application, a dialog will appear that prompts you to configure your license. You can either use a local license or a license from a Pitch Floating License Server™. To use a local license, input the license code. To use a server-based license, input the address to the server.

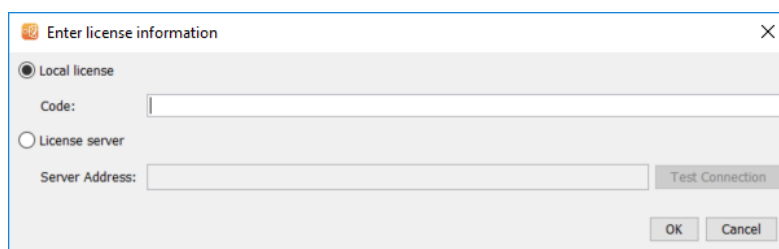


Figure 13. Enter License Key.

If you are using a floating license and the connection to the server breaks while you are working, the application will alert you and allow you to save your work and/or reconfigure your license.

4. An Overview of Pitch Visual OMT™

4.1. Opening a sample project

When opening the application for the first time, a startup screen will be displayed offering the following alternatives for getting started:

- **Open a sample project:** Open one of the sample projects included with the application.
- **Create a new project:** Create a new project from scratch.
- **Open recent project:** Open a recently used project. This will be empty if no projects have been opened before.

4.2. Overview of the GUI

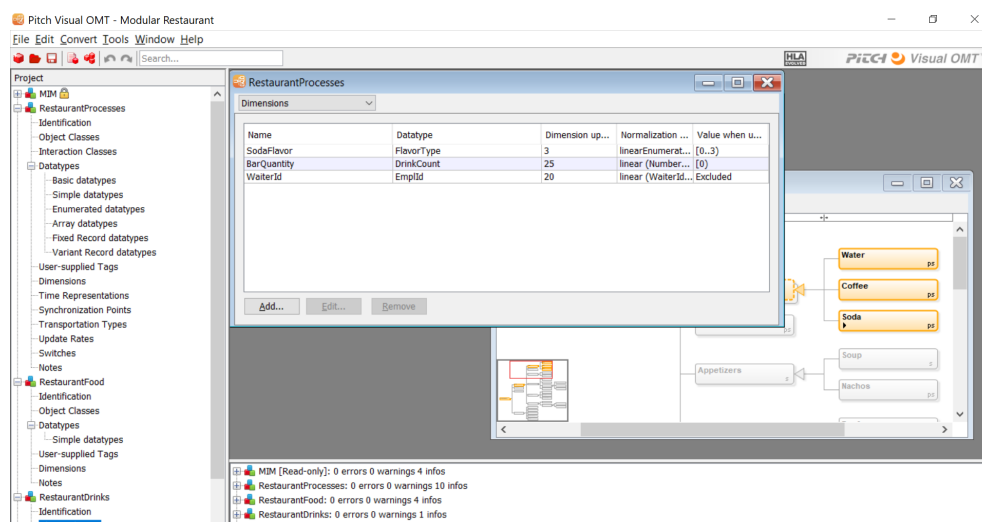


Figure 14. The main window of Visual OMT.

Open the Modular Restaurant sample.

To the left is a tree view of the modules in the project and the standard MIM. To view the tables contained in a module, expand it by clicking [+]. To collapse, click [-].

Double-click a module or table in the project tree to display its window in the main work area (the area with a dark grey background in Figure 14). Windows in the main work area can be closed, moved, resized, minimized and maximized, just like the windows of most standard applications.

In addition to buttons like [Open] and [Save] or [Undo] and [Redo], the toolbar contains a search field which searches the entire project.

Below the main work area, there is another tree view. This view, explained further in

[Section 4.7](#), presents the results of the consistency check performed on the project.

At the bottom is a status bar that displays messages from the application. For longer messages, hovering over the status bar will display the entire message.

4.3. Navigating Modules and Tables

When double-clicking a module in the project tree, a window showing that module will appear in the main work area. If there is already a window open for the chosen module, it will be moved to the front rather than a new one being created. To change which of the module's tables is displayed, you can use the window's dropdown menu or simply double-click a table in the project tree. If you want to display multiple windows for the same module, simply right-click on that module (or one of its tables) and choose "Show *Name* in new Window".

4.4. The Module Overview

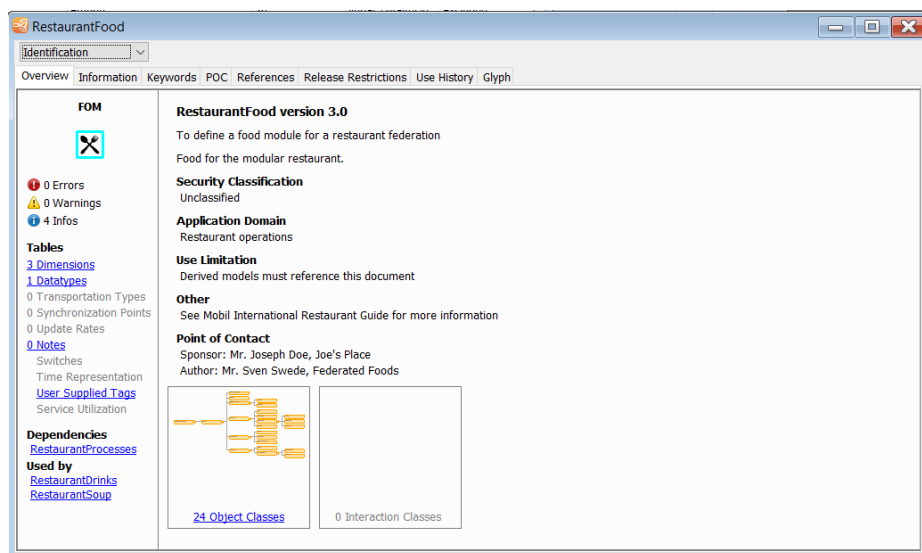


Figure 15. Overview of a module.

The overview tab in a module's identification table provides a short summary of the contents of the module. The type, title, purpose and description give an idea of why the module was written. A feature introduced in HLA Evolved is the possibility to assign a glyph (icon) to each module, which can be seen in the overview window.

The overview window also displays a list of the module's tables and how many elements they contain. Together with thumbnail pictures of the object and interaction class trees, this gives an overview of the contents of the module.

The overview window also contains information about errors, warnings and information messages the module may generate through Visual OMT's consistency check (described further in [Section 4.7](#)). Note that this information depends on the context, and will not

necessarily be the same if the module is opened in another project. For example, warnings about conflicting implementations of a module will only appear in projects where the module leads to these conflicts. There will also be differences in what generates a warning, depending on whether you have enabled the HLA 4 preview features.

4.5. Understanding the Module Class Tree

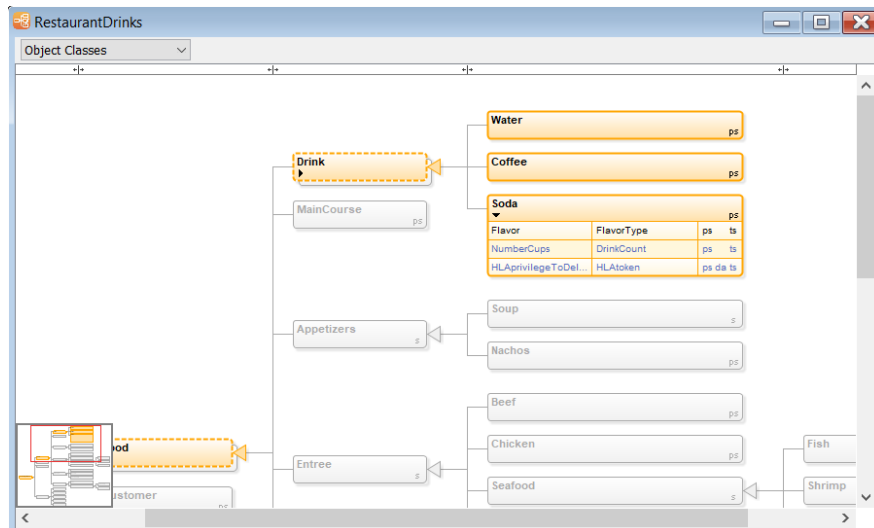


Figure 16. A tree view of object classes.

A module's object classes and interaction classes are shown in a graphical tree, opened through double-clicking them in the project tree to the left of the GUI. This provides a clear overview of how a module's classes are connected.

If the module is dependent on other FOM modules, the trees will be superimposed on grayed-out versions of the classes in the modules it depends on. This makes it easy to see where your classes fit in with other FOM modules. If you wish to hide some or all of this information, simply right-click the background of the tree view and toggle which dependencies are shown. Toggling this does not alter any information in the generated FOMs.

It is also possible to get a list of all attributes of an Object Class (or parameters of an Interaction Class), by clicking the arrow symbol (only visible when applicable). This will not only list the attributes or parameters specified on the class itself, but also those inherited from its parent classes. This also applies for when parent classes are in another module and are only referenced by means of scaffolding, as explained in [Section 6.6](#).

The class tree view includes a minimap that shows a thumbnail view of the class tree. The red rectangle shows the area of the tree that is currently visible. To scroll to a part of the tree using the minimap, drag the red rectangle there or simply click the target area in the minimap. It is possible to hide the map by right-clicking the view and toggling "Show Minimap".

4.6. Datatypes

Name	Datatype	Semantics
UnsignedShort	Basic	Integer in the range $[0, 2^{16} - 1]$
TimeType	Simple	Time representation
LAType	Simple	Time interval (non-negative)
DollarRate	Simple	Cost per hour
Years	Simple	Elapsed time in years
DrinkCount	Simple	Measure of number of drinks
EmplId	Simple	Employee identifier
RateScale	Simple	Evaluation of staff where 1 = best
WaiterTasks	Enumerated	Possible activities of waiters
ExperienceLevel	Enumerated	Level of experience of waiters
FlavorType	Enumerated	Possible flavors of soda
Employees	Array	Identifiers of employees currently working
AddressBook	Array	Collection of all employee addresses
ServiceStat	Fixed Record	Check-off on whether the server performed properly on elements of the meal
AddressType	Fixed Record	Mailing address
WaiterValue	Variant Record	Datatype for waiter performance rating value

Figure 17. Datatype view.

Datatypes and other data are displayed in a tabular format that mirrors the definitions and structure of the OMT standard. Datatypes are edited in dialogs that support you in entering correct and consistent data.

4.7. A Look at the Issue List

The issue list presents all the issues found during the consistency checks automatically performed on the project. Issues can consist of conflicts between different definitions, usage of an undefined datatype, or anything else that is not allowed by the OMT standard. The list will update after every change in the project or any of its modules.

Issues are displayed under the name of the module they are related to. This means that one inconsistency can cause several issues to appear in the tree. For example, if two Dimensions in separate modules have the same name but different upper bounds, this will result in one issue under each affected module.

To the right of each issue description, there are one or more actions. The "Show" action is always present: it will open the location where the issue is caused. Double-clicking on the issue description text has the same effect as clicking the "Show" action. Some types of issues will also have actions for the most common fixes for the issue. Hovering an action will evoke a tooltip describing what changes the action would incur.

For example, in [Figure 18](#) you can see that the note *DocRef* is not used. The action "Remove" is suggested, which would simply remove *DocRef* from the notes table.

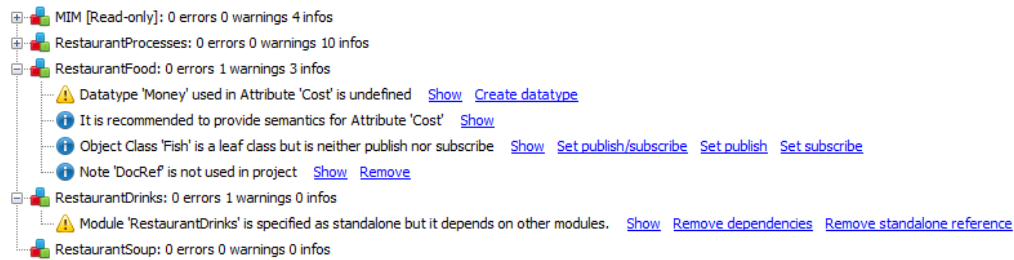


Figure 18. Issue List.

4.8. Searching Elements and Inspecting References

Searching the project is a time-saving feature. It can, for example, be used to quickly find a specific element, or to find where a datatype is being used. The search finds both uses of the element and references to the element in text.

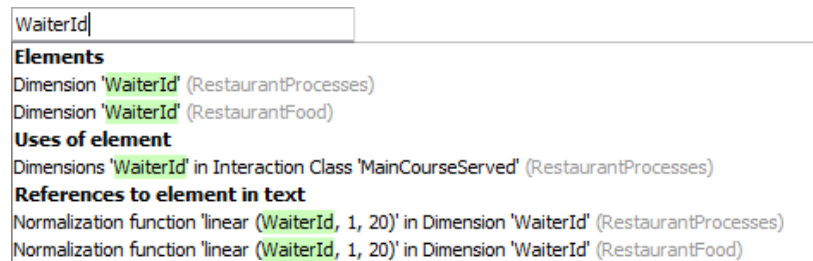


Figure 19. Search field.

4.9. Convenience Features

There are several more features that simplify working with Visual OMT. Among these are drag and drop, which makes it easy to add classes, rearrange them in the class tree view, and move elements between different FOM modules. Cut, copy and paste commands can also be used to move or duplicate data in datatype tables and some other tables.

Undo and redo commands make it possible to undo mistakes and changes that did not have the desired effect. Simply press **Ctrl** + **Z** to undo and **Ctrl** + **Shift** + **Z** to redo, or use the arrow buttons in the header panel.

Throughout the application are tooltips that explain the functionality of the application, and help you understand different fields and what input is allowed.

4.10. Import and Export

Using the "Convert" menu, Visual OMT can import and export to and from HLA 1516-2000 OMT data, HLA 1.3 OMT data and HLA 1.3 FED data. This means that it can be used for editing your FOM regardless of which HLA version you are working with. For details about importing and exporting to and from specific versions of the HLA standard, see [Section 9.6](#), [Section 9.7](#), and [Section 9.8](#).

4.11. Tools

The Project Overview shows a comprehensive, aggregated view of the modules to make it easier to get into a project. It also displays module dependencies and a summary of all modules' content. It also shows all object and interaction classes in a simplified and aggregated class tree. No data can be modified in this view.

For more details of the available tools, see [Section 9](#).

5. Setting up a Project

5.1. Before you Start

Before you start working with Visual OMT, it is recommended that you have a basic understanding of the HLA OMT format. See [Section 2.6](#) for info on further reading.

5.2. Understanding Allowed Dependencies

A module is dependent on another module if it uses elements, like datatypes or transportation types, from the other module. For more info, see [Section 2.3](#).

5.3. Creating a Project

To create a new project, simply select the "New Project..." option in the "File" menu. If your current project has unsaved changes, you will be given a chance to save them before the new project is created.

The Project Properties dialog will be shown after creating a new project. See [Figure 20](#).

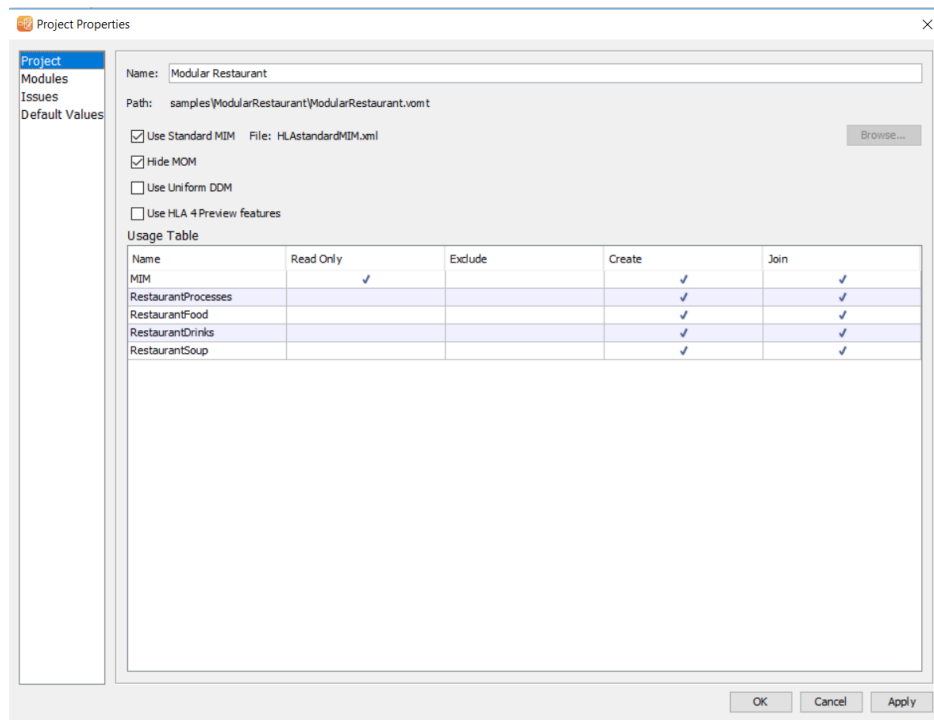


Figure 20. The Project Properties Dialog.

At the top of this dialog is an option to change the name of the Project. Just below it is the path to the project file.

The "Use Standard MIM" checkbox is selected by default. If you have a custom MIM you want to use, uncheck the checkbox and then click [**Browse...**] to select your custom MIM.

Using a custom MIM is an advanced use case, and should only be done when fully understood.

The "Hide MOM" checkbox is also selected by default. When selected, it will hide all MOM data. For example, *HLAmanager* and its subclasses will not be shown in the Object Classes view.

For information on the "Use Uniform DDM" option, see [Section 9.12.3](#).

The "Use HLA 4 Preview features" checkbox is not selected by default. When selected, it will enable HLA 4 preview features like extended FOM merging. You can read about all HLA 4 Preview features in [Section 2.5](#).

Finally, the Usage Table gives a quick overview of how the modules in your project are used. To edit the values in the table, go to the "Module" section.

5.4. Adding a module

There are three ways to add a module to a project:

- Creating a new empty module
- Opening an existing module from file
- Importing a module from a FOM that uses an old OMT format

Creating a new module or editing the settings of an existing module is handled in the "Modules" section of the Project Properties dialog, see [Figure 21](#). For information on how to import a module, see [Section 9.6](#).

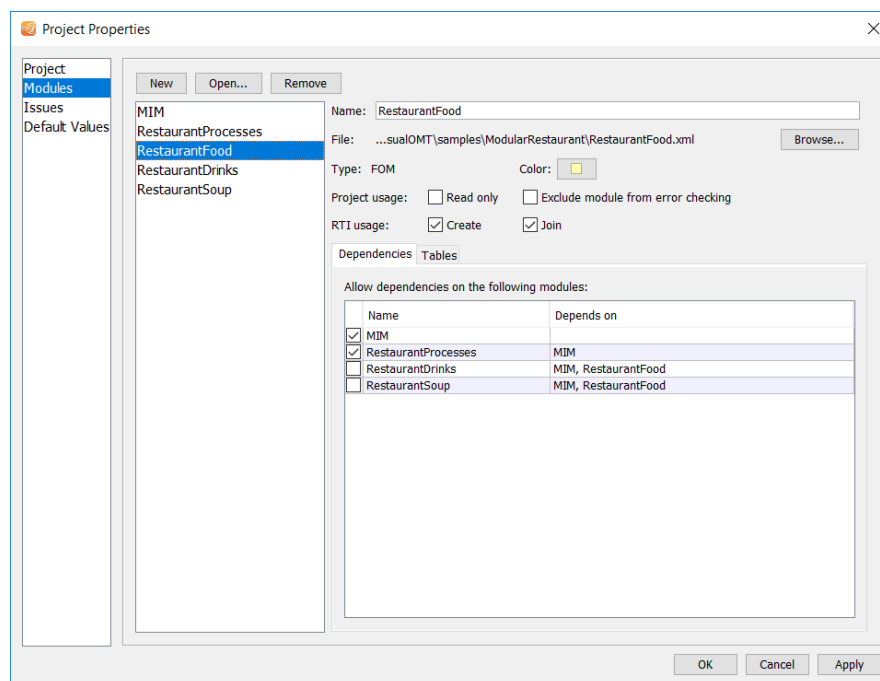


Figure 21. Module settings.

The **[New]**, **[Open...]** and **[Remove]** buttons are used for creating a new module, opening an existing module and removing a module from the project, respectively.

To edit the properties of a module, select it in the list. The following options are available for each module:

- *Name* - This will be used to reference the module in, for example, the Project Tree and the Issue List.
- *File* - Where the module will be saved. Click **[Browse...]** to select a new location.
- *Type* - Whether this module is a FOM or a SOM, as specified in the Identification Table (see [Section 5.5](#)).
- *Color* - The color that will be used to represent this module in Visual OMT.
- *Read-only* - A read-only module cannot be modified. Some uses are mentioned in [Section 2.3](#).
- *Exclude module from error checking* - Selecting this option for a module will make the Issue List ignore it. This is useful for modules that you have added to your project only temporarily: for example, when copying datatypes from an old FOM module.
- *Create* - Select this if the module is used for creating a federation.
- *Join* - Select this if the module is used to join a federation.

Note that a module can be used both for creating a federation and joining one.

In the lower right corner of the Project Properties dialog is an area with two tabs, Dependencies and Tables.

5.4.1. The Dependencies Tab

In the Dependencies tab, you select which modules your module should depend upon. Note that if the module *A* depends on the module *B*, module *A* will implicitly receive *B*'s dependencies. This means that if *B* is dependent on a module *C*, *C*'s datatypes can be accessed from *A* without explicitly defining a dependency from *A* to *C*. Also note that a module's secondary dependencies are visible in this view. [Figure 21](#) shows that RestaurantDrinks is dependent on RestaurantFood, which in turn is dependent on RestaurantProcesses.

5.4.2. The Tables Tab

In the Tables tab, you can add and remove tables from your module. Tables can also be added by right-clicking the module name in the Project Tree, and removed by right-clicking the table name.

5.5. The Overview and the Identification table

The Identification table will be opened in the main work area after adding a new module to your project. The first tab in this table is the Overview tab. It is not possible to edit the information displayed in this tab directly, as the information is gathered from different sources and only displayed here as a convenient way to quickly get an understanding of the module.

The second tab is the Information tab (see [Figure 22](#)), which has the following fields:

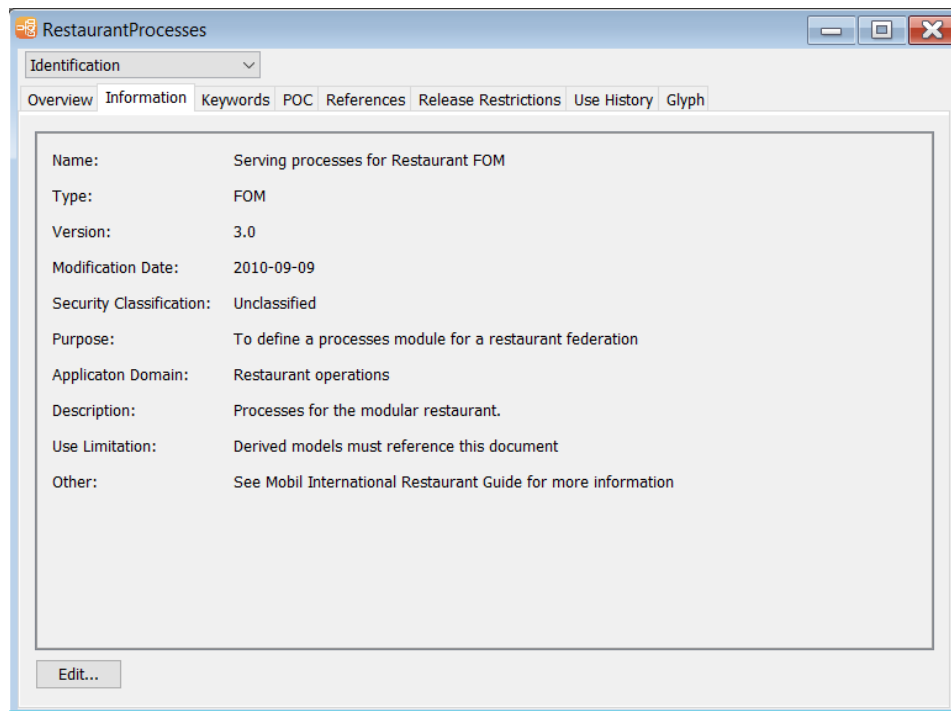


Figure 22. The information tab in the identification table.

- *Name* - Name of this module. This is the name that will appear in the <name> tag in the generated FOM files.
- *Type* - The module's type. This can be either FOM or SOM.
- *Version* - The current version of this module.
- *Modification Date* - When the module was last modified.
- *Security Classification* - The security classification of this module.
- *Purpose* - The purpose for developing this module.
- *Application Domain* - The type or class of application to which this module applies, such as analysis or training.
- *Description* - A description of the content of this module.
- *Use Limitation* - Known constraints on the application of this module.
- *Other* - Other information useful for the users of this module.

The Keywords tab (see [Figure 23](#)), is used for associating keywords with the module. When adding a keyword, the taxonomy field is optional. The taxonomy may specify the source of the keyword vocabulary. For example, taxonomy might be "Food Service Industry" and the keyword might be "Waiter".

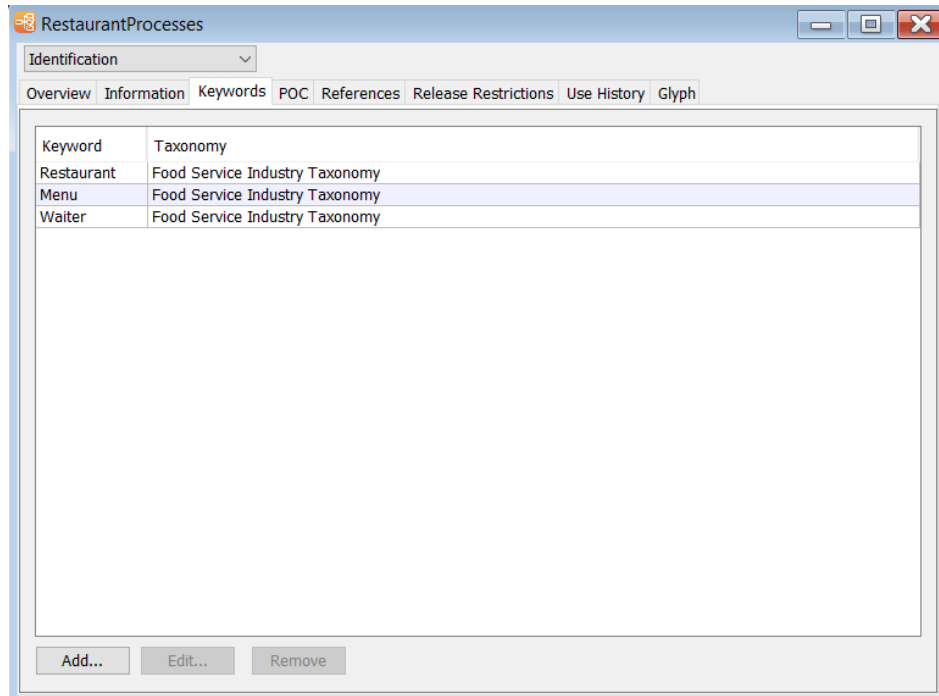


Figure 23. The Keywords tab in the Identification table.

The POC tab (see [Figure 24](#)) lists the points of contact for this module. The following information can be specified when adding a POC to the module:

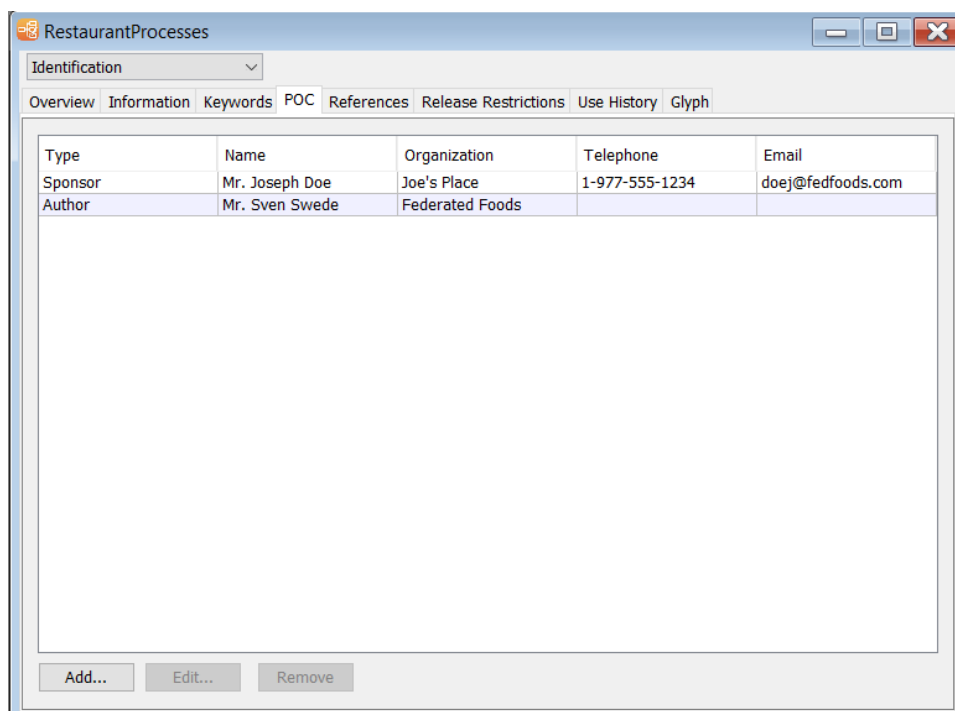


Figure 24. The POC tab in the Identification table.

- *Type* - The type of this POC; for example, "Primary author" or "Sponsor"
- *Name* - Name of this POC
- *Organization* - The organization the POC belongs to, if any
- *Telephone* - More than one telephone number can be specified for each POC
- *Email* - More than one email can be specified for each POC

The Reference tab (see [Figure 25](#)) is used to specify pointers to additional sources of information.

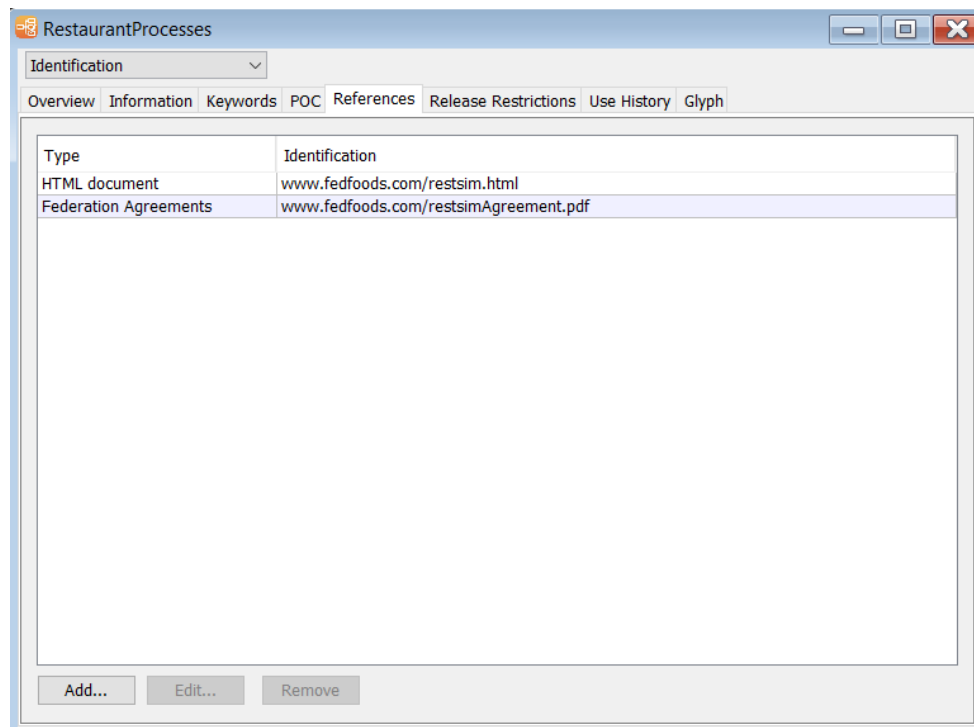


Figure 25. The References tab in the Identification table.

When adding a reference, the first thing to specify is the type of the new reference. The kind of information that should be entered in the Identification field depends on the type of the reference. Help for entering the correct information is given in the Add reference dialog.

The Release Restrictions tab (see [Figure 26](#)) lists any restrictions on the release of this module to specific organizations or individuals.

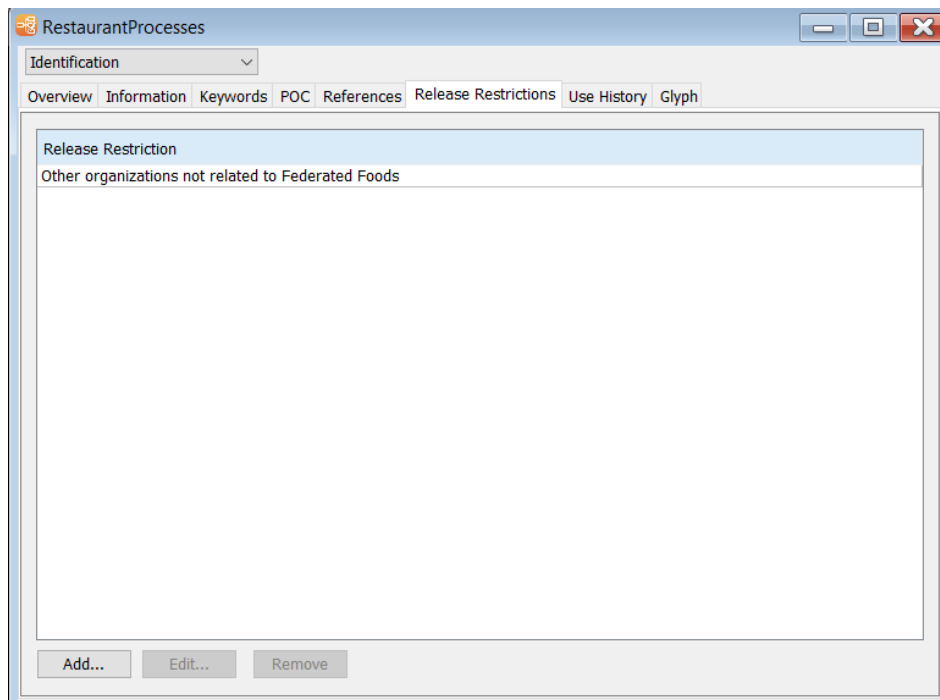


Figure 26. The Release Restriction tab in the Identification table.

The Use History tab (see [Figure 27](#)) lists past applications of this module.

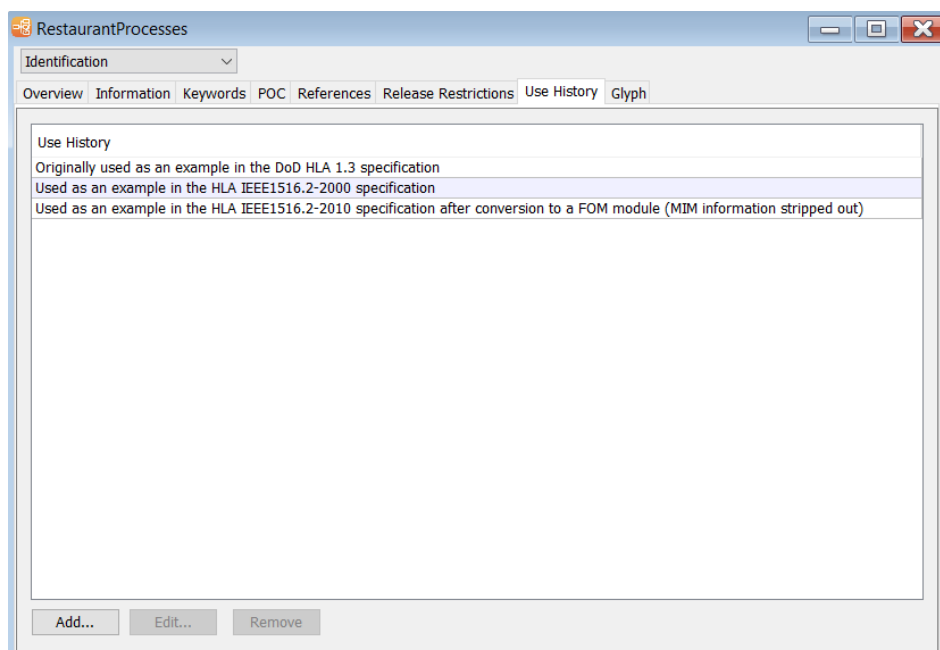


Figure 27. Use History tab in the Identification table.

The Glyph tab (see [Figure 28](#)) specifies a glyph if one has been associated with this module.

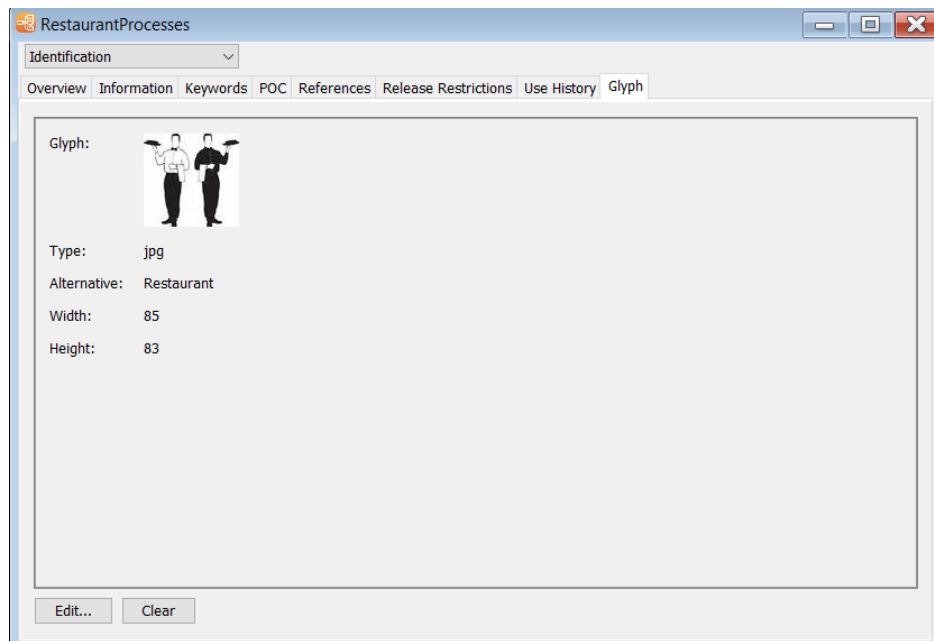


Figure 28. Glyph tab in the Identification table.

If a glyph has been assigned, it can be seen in the Glyph tab (it is also visible in the Overview tab). Also visible is the type, i.e., the file format, as well as an alternative text description of the glyph and the width and height in pixels.

When adding a glyph, Visual OMT will try to fill in the Type, Width and Height fields for you. This is only possible if your glyph uses a well-known format. Using a lesser known format could give other users of the FOM problems displaying the glyph correctly. To avoid this kind of problem, we recommend that you use a glyph of size 32 by 32 pixels in the GIF, JPEG or PNG formats.

5.6. Other Project settings

The Project Properties dialog is also used to select what to display in the Issue List; see [Figure 29](#). In the Issues section, you can select Show errors, Show warnings and/or Show info. Only the selected levels will be shown in the Issue List.

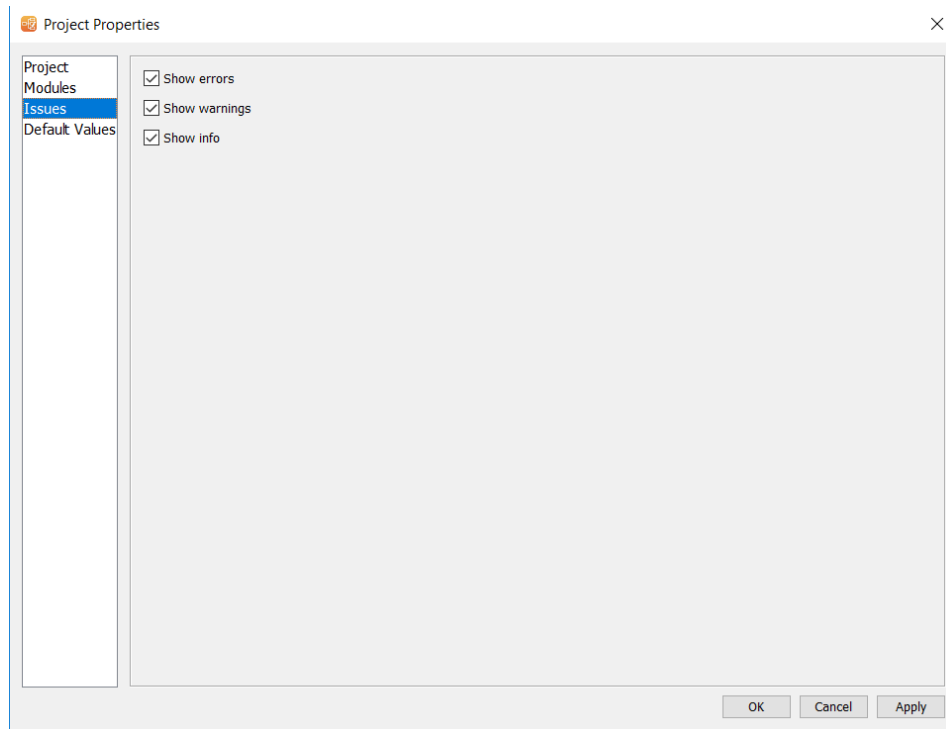


Figure 29. Specifying what to display in the Issue List.

The last section in the Project Properties dialog is Default values; see [Figure 30](#).

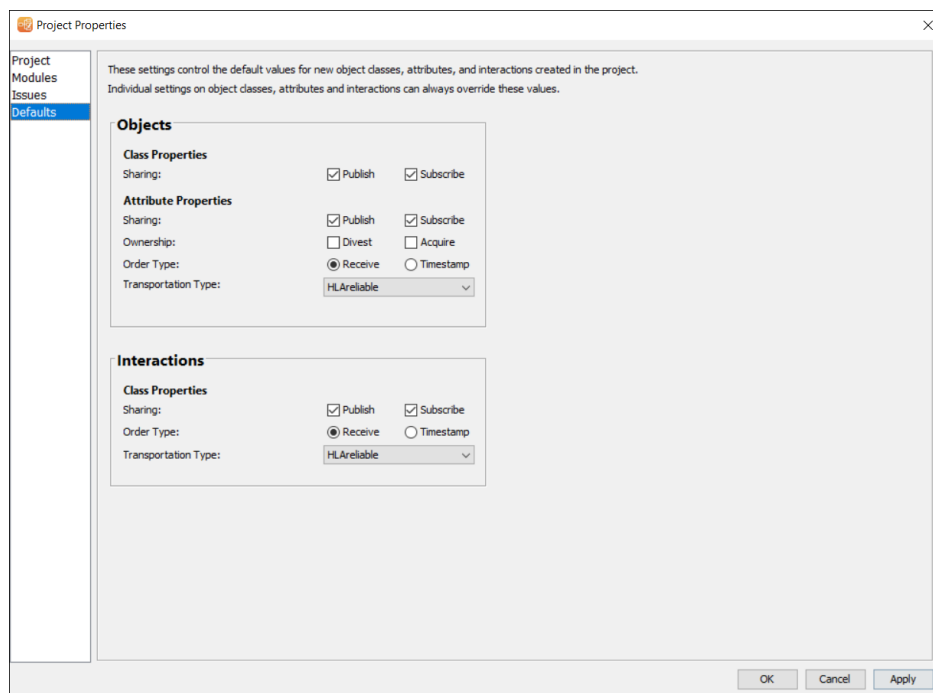


Figure 30. Default value settings.

The default values you enter here will be used for initial values in newly created attributes, interaction and object classes.

6. Working with Class Trees

This section describes object and interaction class trees, which contain either one node for each of a module's object classes, or one for each of its interaction classes. The purpose of a class tree is to visualize the hierarchy of a module's classes.

To add an object or interaction class to a module, open the corresponding class tree from the project tree (left side of the application) and right-click the class that you want as the parent class. Select "Add Object Class" (or "Add Interaction Class"). In the same menu, there are options to rename the class and to delete it. Additionally, you can move a class in the tree by right-clicking it and selecting "Move Up" or "Move Down". To change a class's position in the hierarchy, use the drag and drop feature.

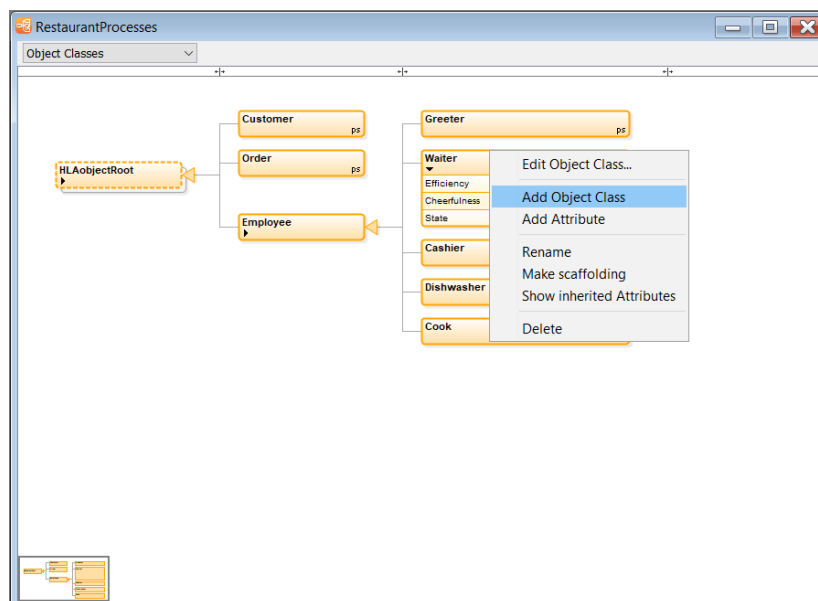


Figure 31. Simple Object Class tree.

6.1. Drag and drop within a module

To change a class's position in the hierarchy, simply grab it and drag it to its new destination. As you drag the class around the tree view, a gray line shows which class would be its parent if you were to drop it at its current position. If you drop it in the wrong place, you can always use the undo functionality.

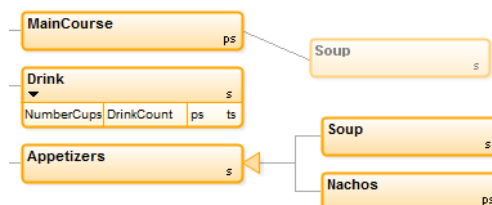


Figure 32. Drag and drop of an Object Class.

You can also use the drag and drop feature to add new object or interaction classes to a

module: dragging from near the rightmost edge of an existing class and dropping anywhere within the opened window will result in a new child class.

6.2. Drag and drop between modules

The drag and drop feature allows for quick and easy copying of classes from one FOM module to another. To use this feature, open up either the object class trees or the interaction class trees of two different modules. Move or resize the windows so that they are both visible. Then, simply drag classes from one window to the other. Just like when restructuring *within* a module, there will be a grey line indicating the class's parent-to-be. Note that this creates a copy of the dragged class; the class will also remain in the original module.

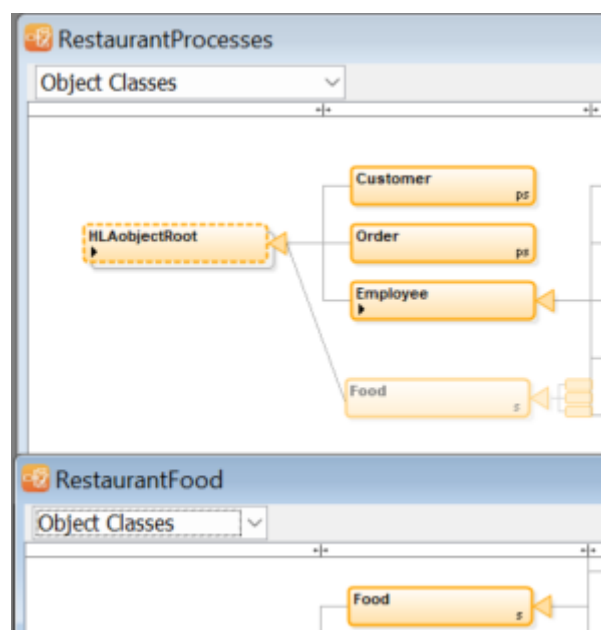


Figure 33. Drag and drop of the Food class from RestaurantFood to RestaurantProcesses.

6.3. Object Classes and Attributes

Right-click an object class and select "Edit Object Class..." to open the Edit Object Class dialog (Figure 34) where all data on the object class can be edited.

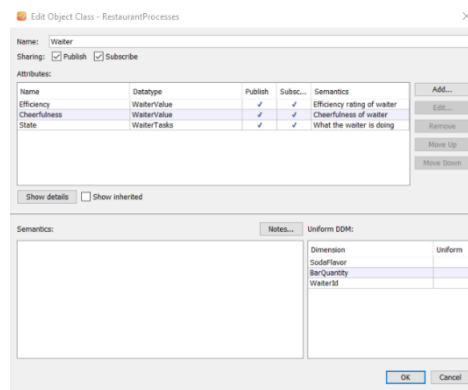


Figure 34. Object Class dialog.

To add an attribute to an object class, simply click **[Add...]** in the object class dialog. This will open the Edit Attribute dialog (Figure 35). To edit existing attributes, simply double-click them in the list or select them and click **[Edit...]**.

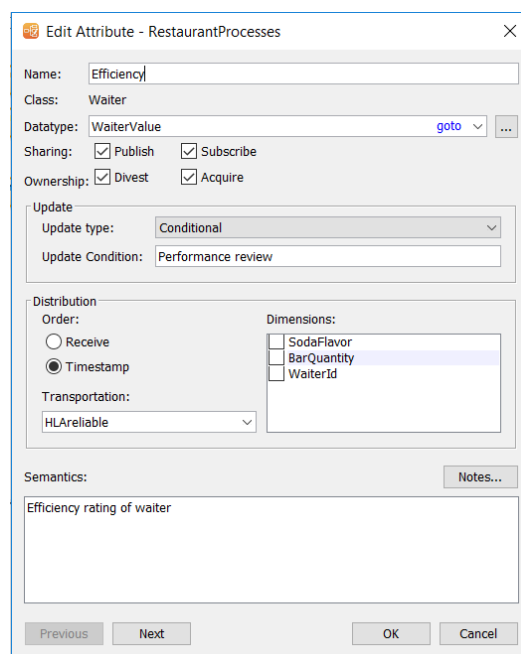


Figure 35. Edit Attribute dialog.

The controls of the dialog are designed to help you enter valid input: the checkboxes, radio buttons and drop-down list make it clear which values are possible and eliminate the risk of typos.

There are often many datatypes to choose from. Click **[...]** beside the datatype field to open the datatype selector. In this dialog, you can filter and sort the available datatypes to find the correct one.

The attributes are displayed in the graph when an object class node is expanded. Expand an object class node by clicking on the small black triangle. The attributes can also be viewed in the object class dialog. To get a complete list of attributes for a class, including those inherited from super classes, right-click on the class and select "Show Inherited

Attributes", or check the box "Show Inherited" in the Object Class dialog.

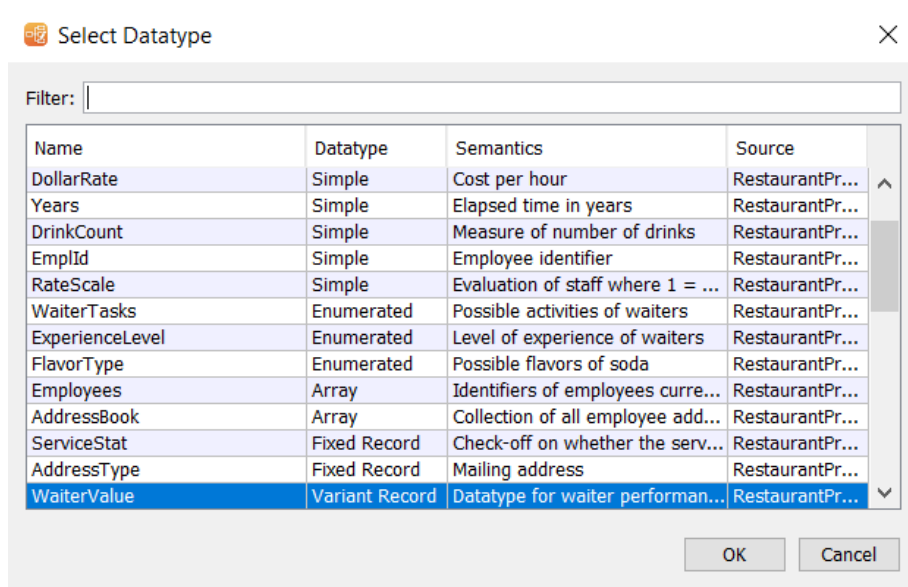


Figure 36. Datatype selector.

6.4. Interaction Classes and Parameters

Adding and editing parameters to interaction classes works in mostly the same way as adding and editing attributes to object classes. As you can see in [Figure 37](#), the main difference is that the Edit Parameter dialog is a lot simpler than the Edit Attribute dialog.

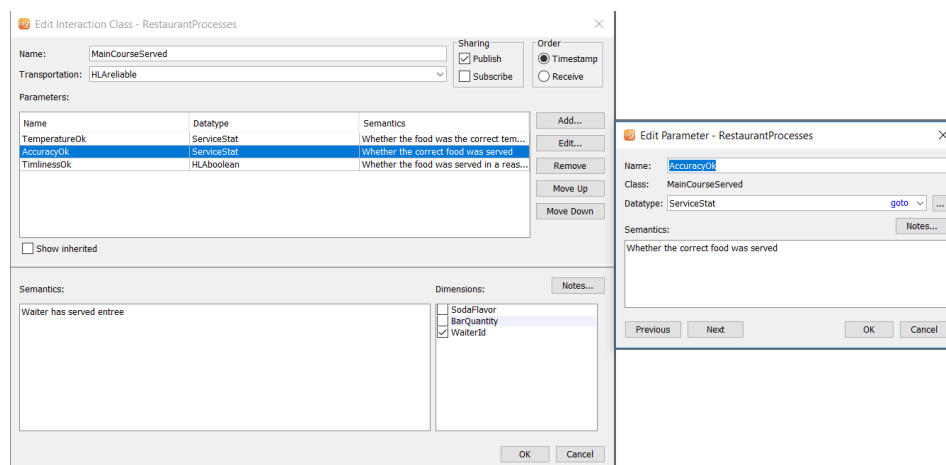


Figure 37. Interaction Class and Parameter dialogs.

6.5. How classes are displayed

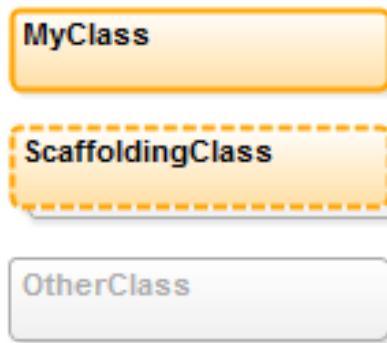


Figure 38. Different types of classes.

A Class belonging to the module currently being edited will be displayed either orange, if it is an Object Class, or green, if it is an Interaction Class. If the Class is not in the current module, but in one of its dependencies, it is displayed as a grayed out version of itself.

If the node is a scaffolding definition (further explained in [Section 6.6](#)), it is displayed with a dashed border.

Waiter		ps
Efficiency	WaiterValue	ps da ts
Cheerfulness	WaiterValue	ps da ts
State	WaiterTasks	ps da ts
PayRate	DollarRate	ps da ts
YearsOfService	Years	ps da ts
HomeNumber	HLAASCIIstring	ps da ts
HomeAddress	AddressType	ps da ts
HLAprivilegeTo...	HLAtoken	ps da ts

Figure 39. Different types of attributes.

When nodes are stacked on top of each other, it means that there are several definitions of the class in separate modules.

Attributes are shown in different colors. Attributes defined on the class are shown in black, and inherited attributes are shown in blue. Attributes that are grayed are defined in other modules.

The name and datatype of the Attribute is displayed, together with letter combinations representing important information about the attribute. "ps" means Publish/Subscribe, "da" means Ownership Divest/Acquire, and "ro"/"ts" means Receive/Timestamp Order.

6.6. Scaffolding and Dependencies

By default, the class tree displays the composition of the current module and its dependencies. As detailed in [Section 6.5](#), classes with dashed borders are scaffolding classes. These classes have a complete definition in a dependency module but also a partial definition in the dependent module. In the FOM, a scaffolding class may be an

object class that only has a name. This inclusion of class hierarchy from dependency modules in the dependent module is useful when creating subclasses or copying full class definitions.

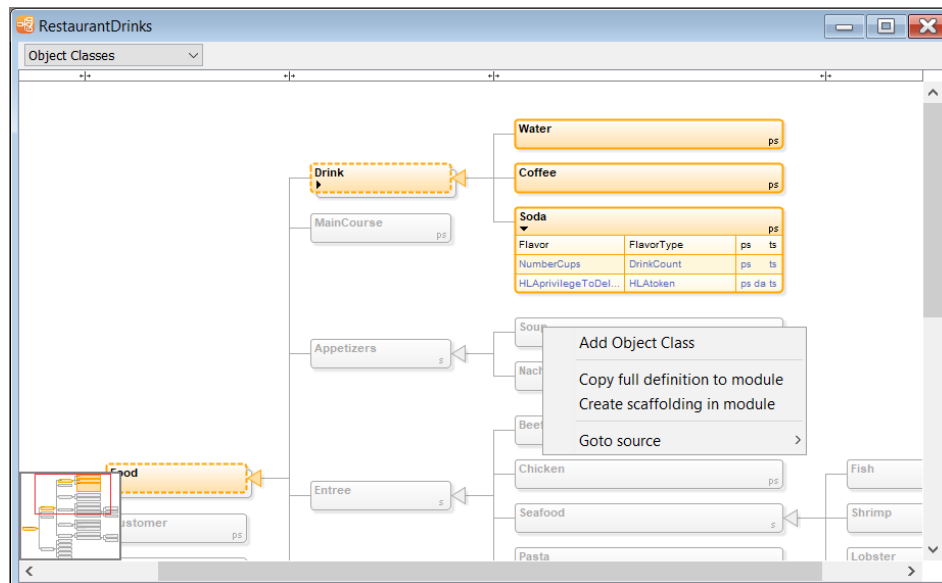


Figure 40. Class tree with composite of current module and dependencies.

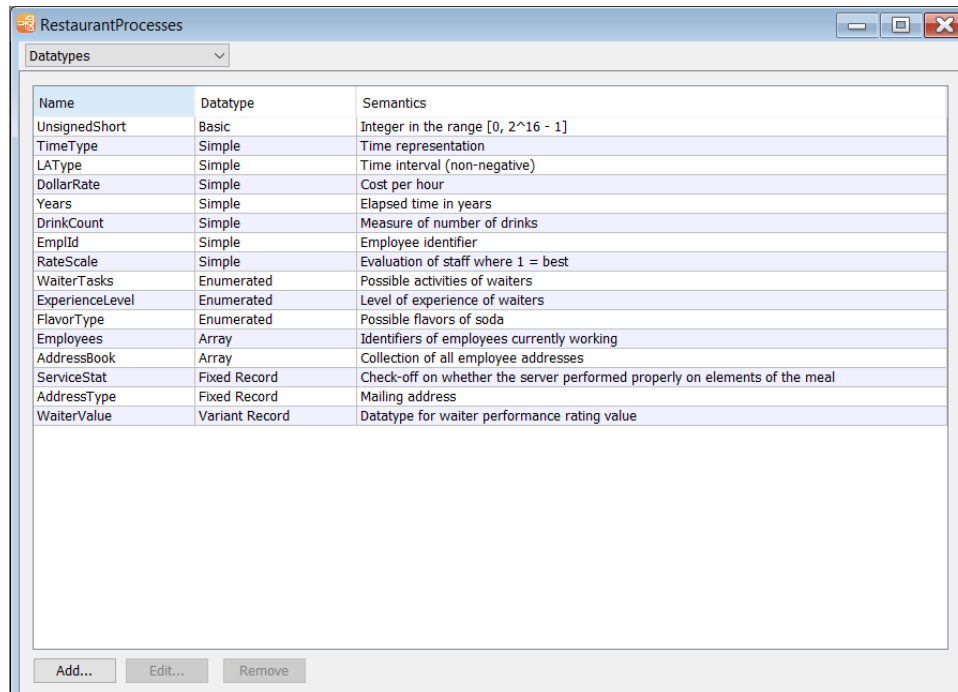
Right-clicking on a class in a different module shows a menu with a few available options. "Add Object Class" will create a chain of scaffolding classes down to the selected class, and then add a new sub class to it. In the example above, it would create a scaffolding class for Appetizers, another one for Soup, and a new sub class under Soup. "Copy full definition to module" will create a chain of scaffolding classes down to the parent of the selected class, and then copy a full definition of the selected class to the current module. "Create scaffolding in module" will create a chain of scaffolding classes down to the selected class.

When your current module is dependent on modules with big class trees, the composite tree can be cumbersome to navigate. Right-clicking the view (outside any class node) and hovering "Show Dependencies" reveals a menu where you can select which dependencies to show and which to hide in the tree view. Visual OMT saves these GUI preferences, but they don't affect generated FOMs.

7. Working with Datatypes

7.1. Datatype overview

The datatype table shows all datatypes in the module regardless of their type. As in all tabular views, the contents can be sorted by clicking on the table headers.



Name	Datatype	Semantics
UnsignedShort	Basic	Integer in the range [0, 2 ¹⁶ - 1]
TimeType	Simple	Time representation
LAType	Simple	Time interval (non-negative)
DollarRate	Simple	Cost per hour
Years	Simple	Elapsed time in years
DrinkCount	Simple	Measure of number of drinks
EmplId	Simple	Employee identifier
RateScale	Simple	Evaluation of staff where 1 = best
WaiterTasks	Enumerated	Possible activities of waiters
ExperienceLevel	Enumerated	Level of experience of waiters
FlavorType	Enumerated	Possible flavors of soda
Employees	Array	Identifiers of employees currently working
AddressBook	Array	Collection of all employee addresses
ServiceStat	Fixed Record	Check-off on whether the server performed properly on elements of the meal
AddressType	Fixed Record	Mailing address
WaiterValue	Variant Record	Datatype for waiter performance rating value

Figure 41. Datatype table.

There are two ways to add a datatype. Pressing "Add" in the general datatypes view and then choosing the type, or pressing "Add" in the table view for that datatype. When the datatype is created, a dialog is shown where information about the datatype can be entered.

To change a datatype, double-click it in the table, or select it and click **[Edit]**, to see the same dialog again.

To move a datatype from one module to another, either select it and use the cut, copy and paste commands, or use drag and drop to move it to another table. Drag and drop for datatypes works the same way as for classes ([Section 6.2](#)): open one datatype table for each relevant module, move or resize them to make them both visible at the same time, and you're good to go. For drag and drop to work, both tables need to be of the same type ("Simple datatypes", "Enumerated datatypes", etc).

To remove a datatype, select it in the table and click **[Remove]** or press the **Delete** key.

7.2. Basic Datatypes

Basic datatypes can be edited and deleted, but new basic datatypes cannot be added. If you want other basic types than those available in the standard MIM, consider defining a custom datatype (for example, a Fixed Record or an Array) using a combination of standard datatypes instead.

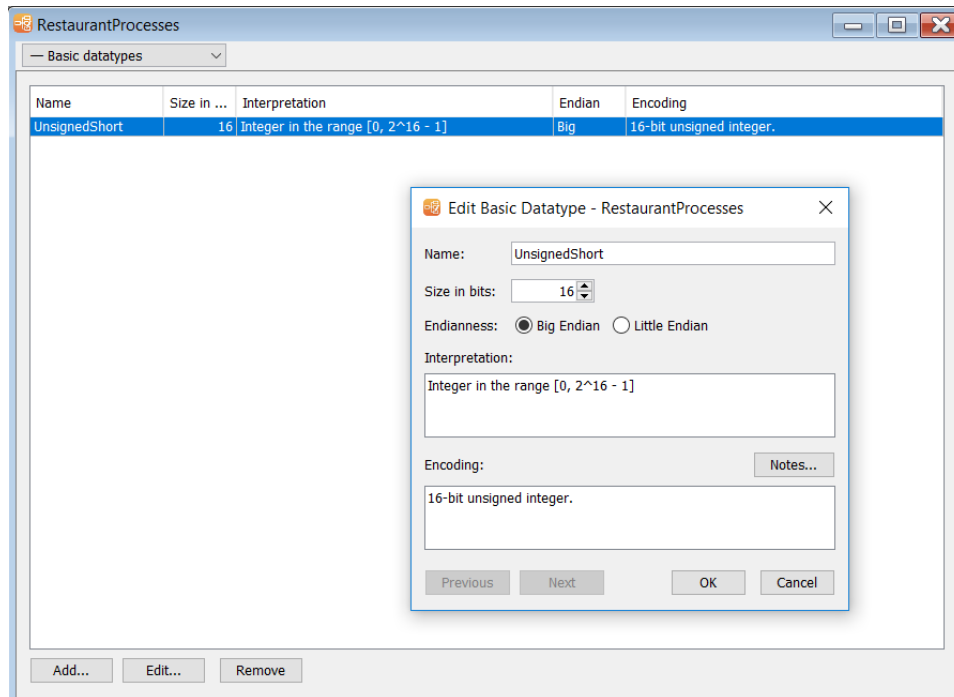


Figure 42. Basic datatypes table and dialog.

7.3. Simple Datatypes

The representation for a simple datatype must be a basic datatype. Hence, only basic datatypes are available in the drop-down box.

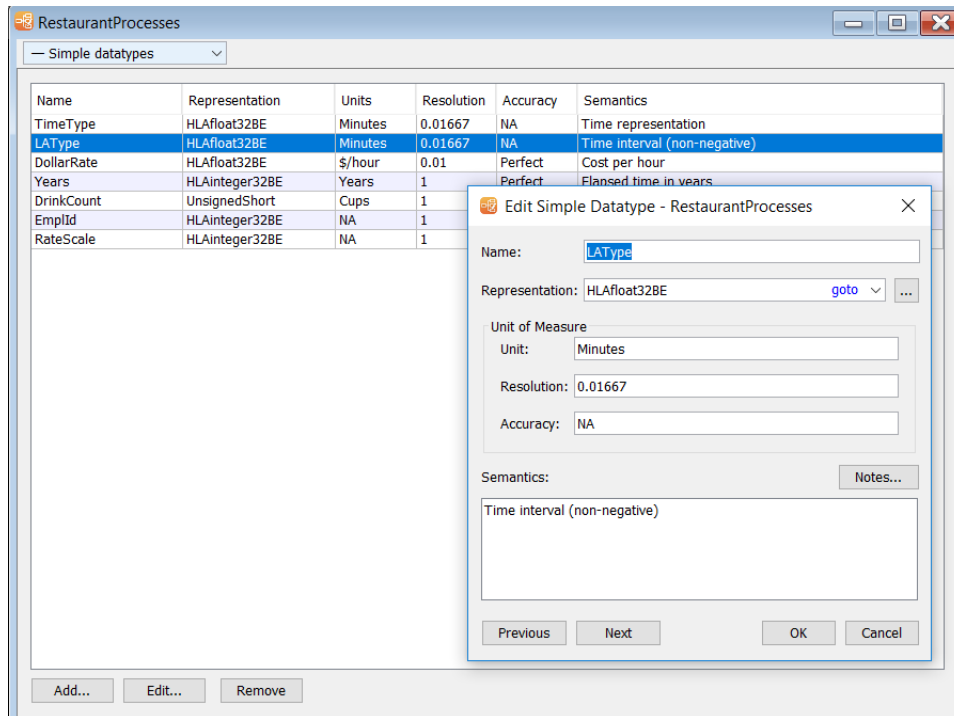


Figure 43. Simple datatypes table and dialog.

7.4. Enumerated Datatypes

The representation for an enumerated datatype must be a basic datatype. Hence, only basic datatypes are available in the drop-down box. To manage enumerators in your datatype use the **[Add]**, **[Edit]** and **[Remove]** buttons.

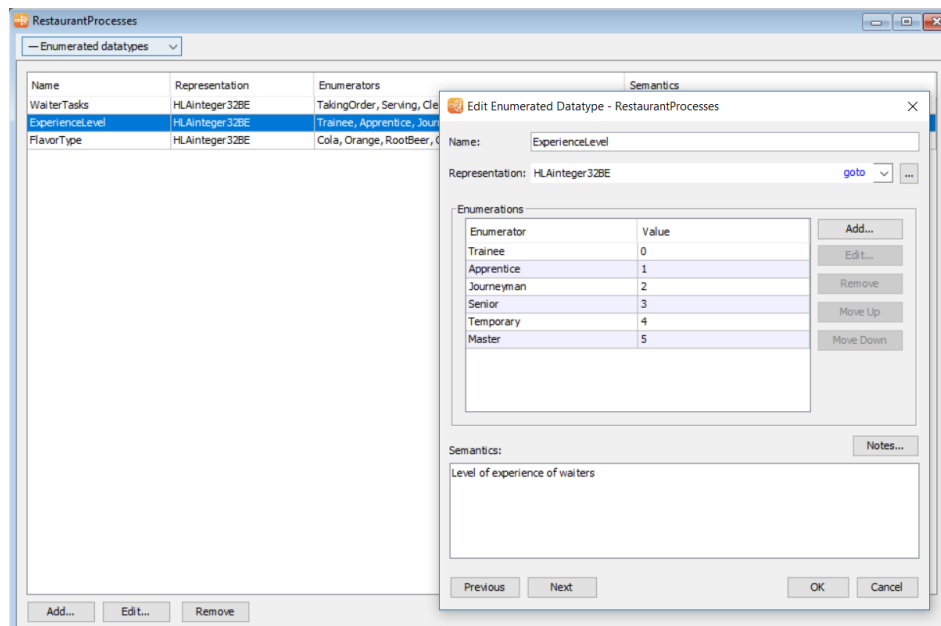


Figure 44. Enumerated datatypes table and dialog.

7.5. Array Datatypes

Arrays have either a fixed length or a variable length. Fixed length arrays must have exactly the specified length. Variable length arrays contain a length field encoded as an `HLAinteger32BE`, and may contain zero or more elements. All elements of an array must have the same datatype.

It is recommended to use the standardized `HLAfixedArray` and `HLAvariableArray` encodings and Dynamic cardinality, instead of any user-defined cardinality or encoding.

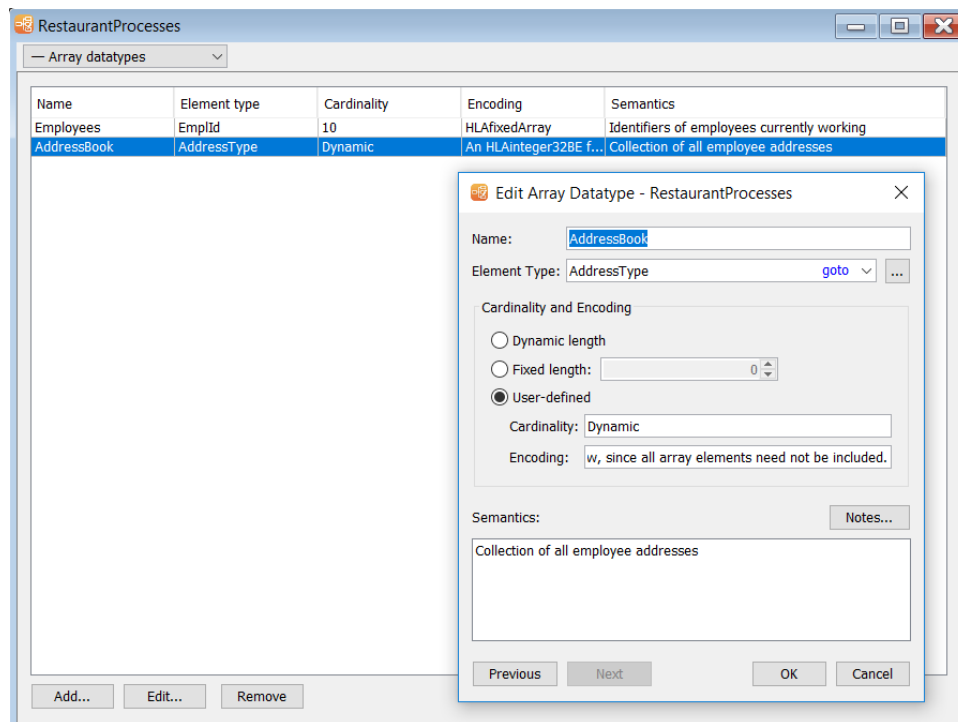


Figure 45. Array datatypes table and dialog.

7.6. Fixed Record Datatypes

Fixed records contain one or more fields, where each field has its own specified datatype. To manage fields in your fixed record, use the **[Add]**, **[Edit]** and **[Remove]** buttons.

It is recommended to use the standardized encoding `HLAfixedRecord`.

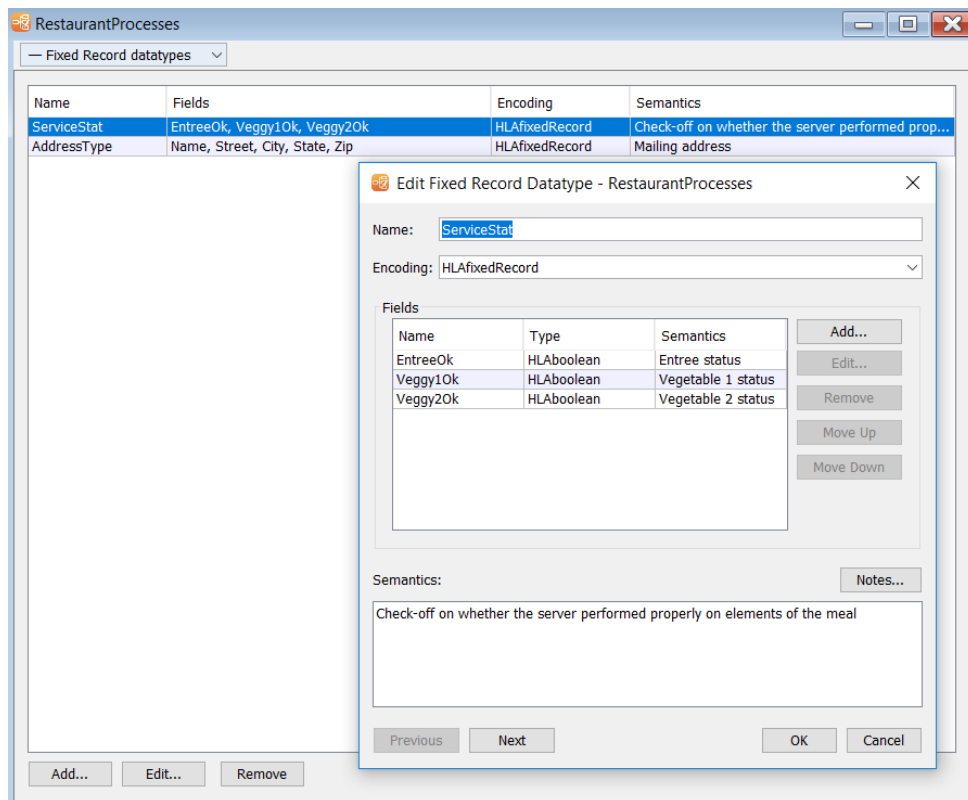


Figure 46. Fixed Record datatypes table and dialog.

7.7. Variant Record Datatype

Variant records have a discriminant that is used to specify which alternative is active (contains information). The discriminant is an enumeration and each alternative is assigned to one or more enumerator values. There is also a special value `HLAOther` to indicate "all other unassigned enumerator values".

It is recommended to use the standardized encoding `HLAvariantRecord`.

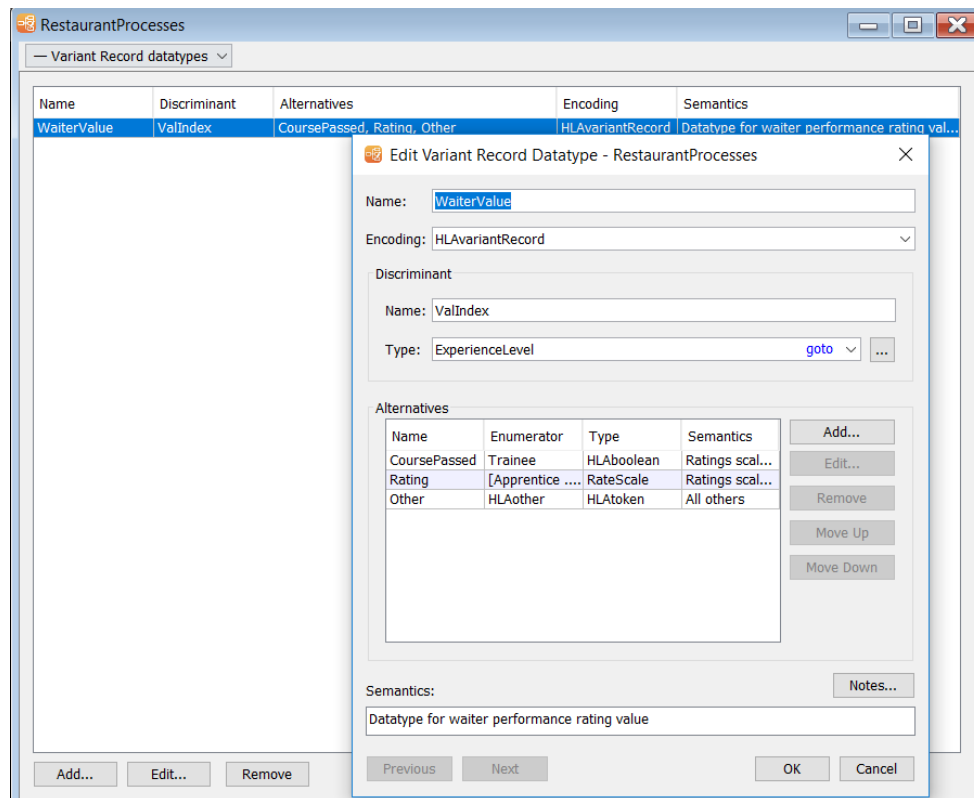


Figure 47. Variant Record datatypes table and dialog.

8. Working with Other Tables

8.1. Synchronization Points

For the Capability field of the Synchronization Point dialog, the allowed values differ between FOM modules and SOM modules. For a FOM, the only allowed value is "NA". For a SOM, the allowed values are "Register", "Achieve", "Register and Achieve" and "Neither Register nor Achieve".

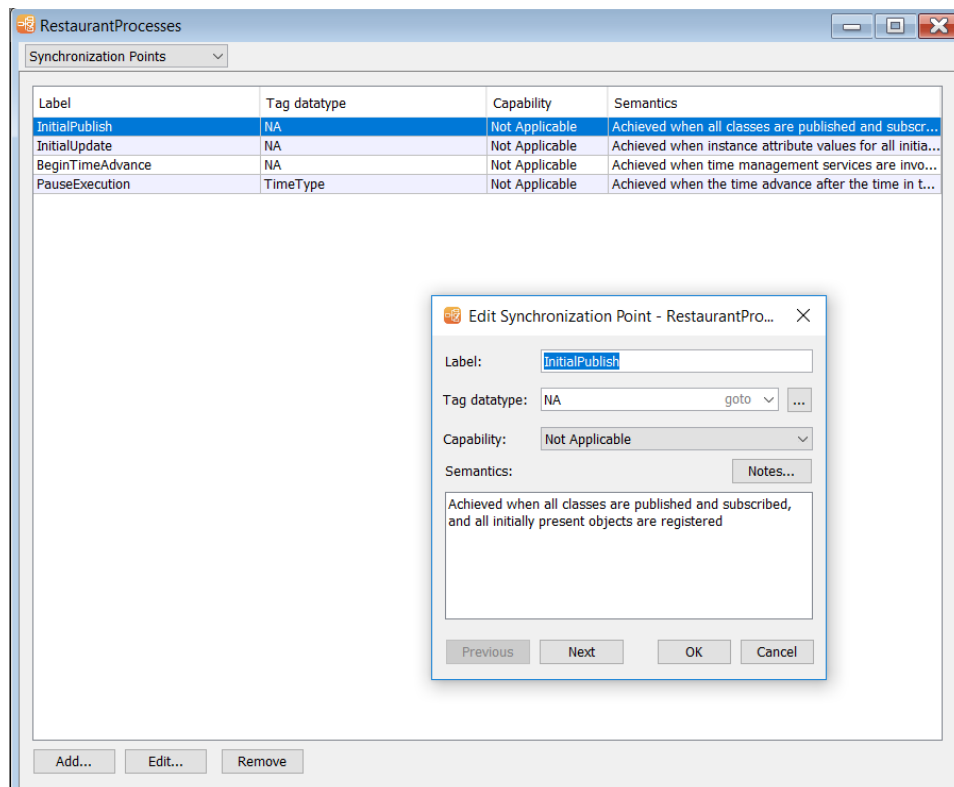


Figure 48. Synchronization Points table and dialog.

8.2. Time Representation

New in HLA Evolved are two datatypes dedicated to expressing time: `HLAinteger64Time` and `HLAfloat64Time`. One of these can be chosen as time representation. Custom types can also be used.

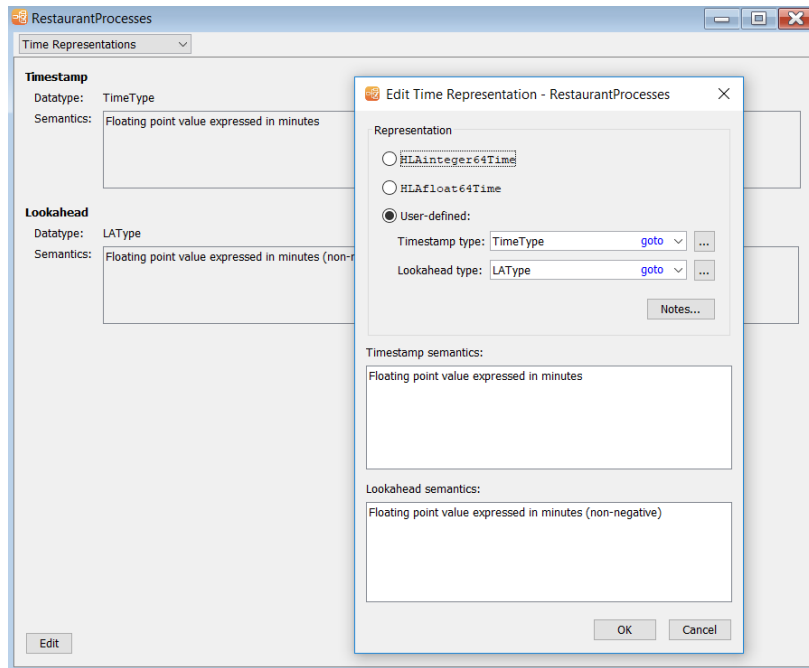


Figure 49. Time Representation view and dialog.

8.3. Dimensions

The datatype of a dimension needs to be either simple or enumerated. Hence, only simple and enumerated types are available in the drop-down list.

A Dimension's "Value when unspecified" can be either "Excluded", an open interval, or a point. If you select "User Defined Value", the value may not be formatted correctly.

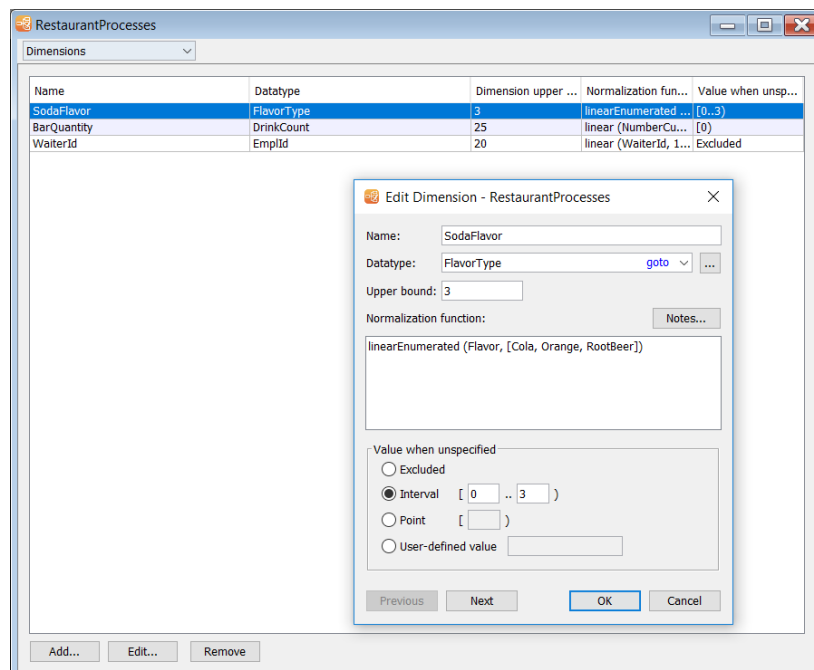


Figure 50. Dimensions table and dialog.

8.4. Transportation Types

Be careful when selecting Reliability for your new Transportation Types, since this is what the RTI will use if it does not support your custom Transportation Type.

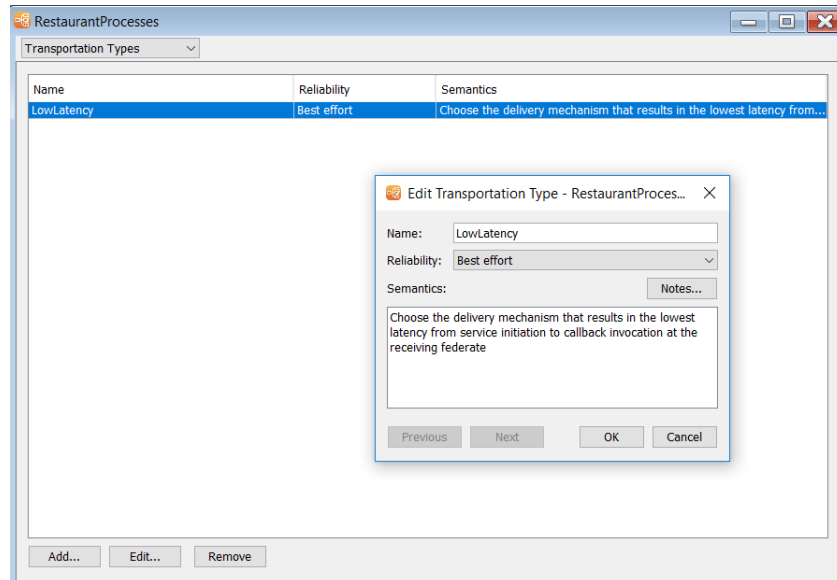


Figure 51. Transportation Types table and dialog.

8.5. User Supplied Tags

User Supplied Tags can be edited in a dialog just like the other types, but there is a fixed number of specified tags and new ones cannot be added.

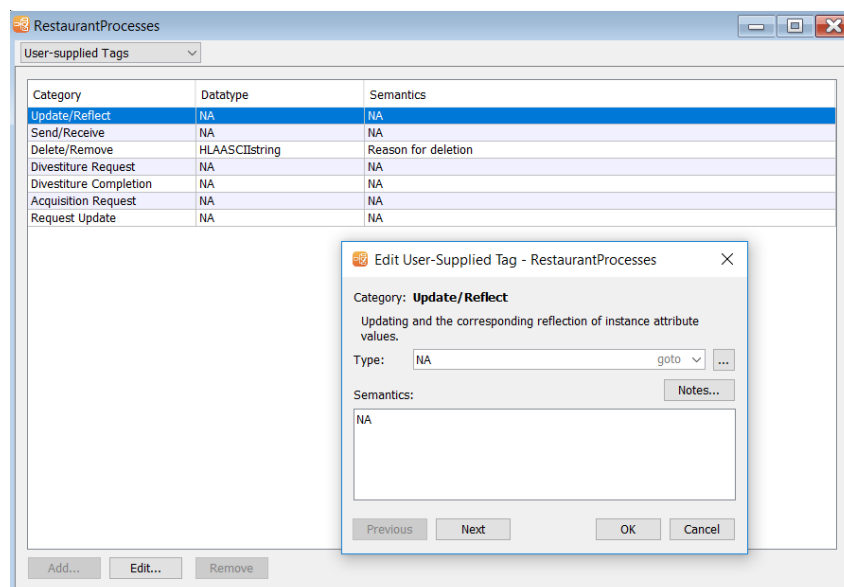


Figure 52. User Supplied Tags table and dialog.

8.6. Update Rates

Specify the Update Rates in the Update Rates table. The unit for *maximum update rate* is *Hz*, i.e. updates per second.

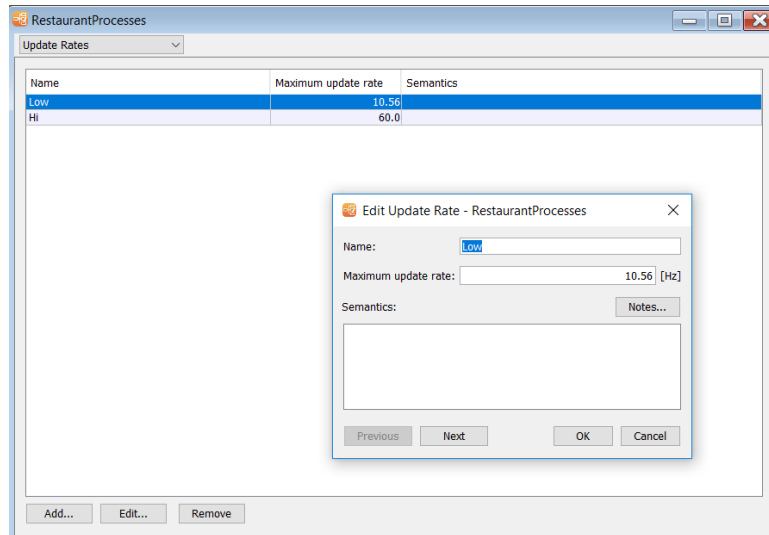


Figure 53. Update Rates table and dialog.

8.7. Switches

In the switches view you will find a checkbox for every switch. Note that at least one of the modules used to create the federation must include the switches table.

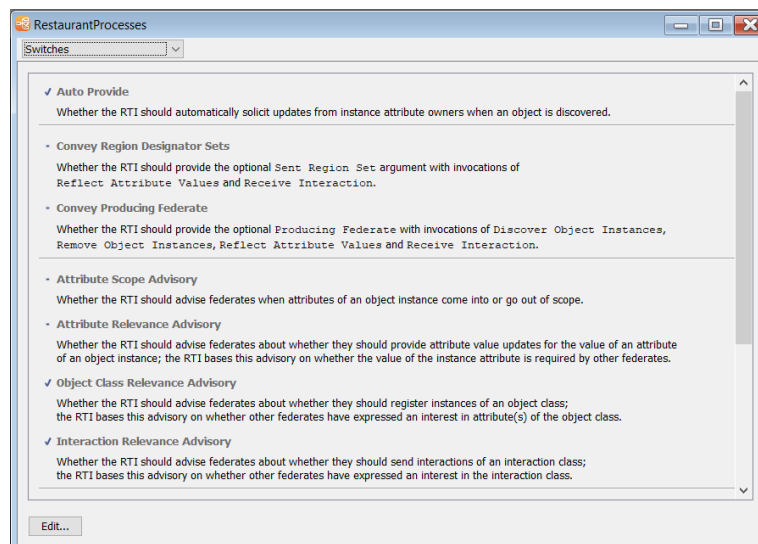


Figure 54. Switches dialog.

8.8. Notes

Notes can be created either in the notes view or as needed when adding notes to elements.

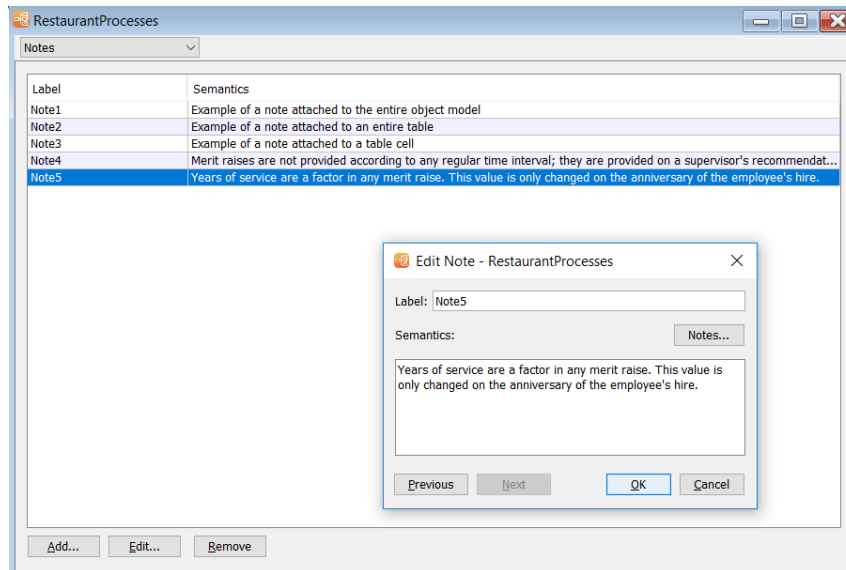


Figure 55. Notes table and dialog.

8.9. Service Utilization

In the service utilization view, the services are presented in a tree, grouped together in service groups. Click [**Edit..**] and check the services your module utilizes. Note that for a SOM, the federation management services "Join Federation Execution" and "Resign Federation Execution" are required, and for a FOM, "Create Federation Execution" and "Destroy Federation Execution" are required as well.

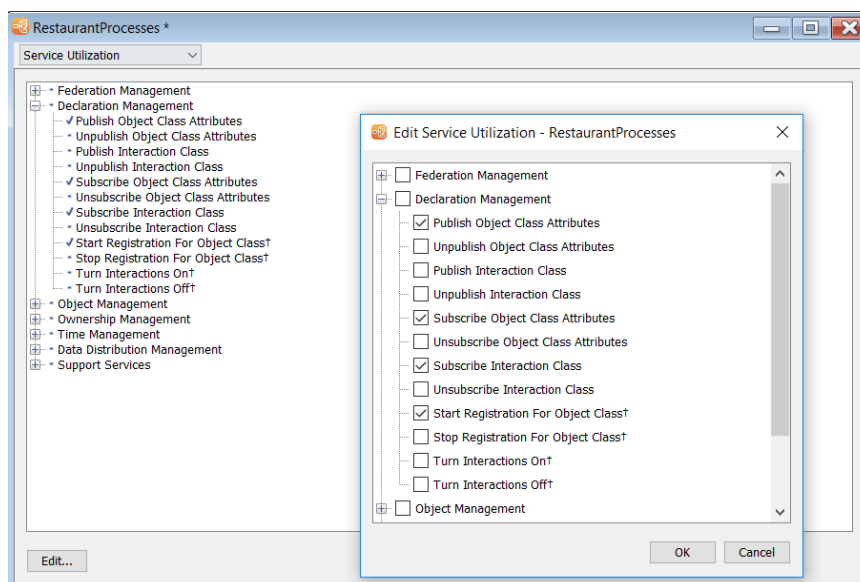


Figure 56. Service Utilization view and dialog.

9. Advanced Topics

9.1. Refactoring

Visual OMT has support for advanced refactoring of FOM modules. One example is *renaming* of elements that are referenced by other elements. The Rename refactoring is supported for datatypes, dimensions, transportation types and notes.

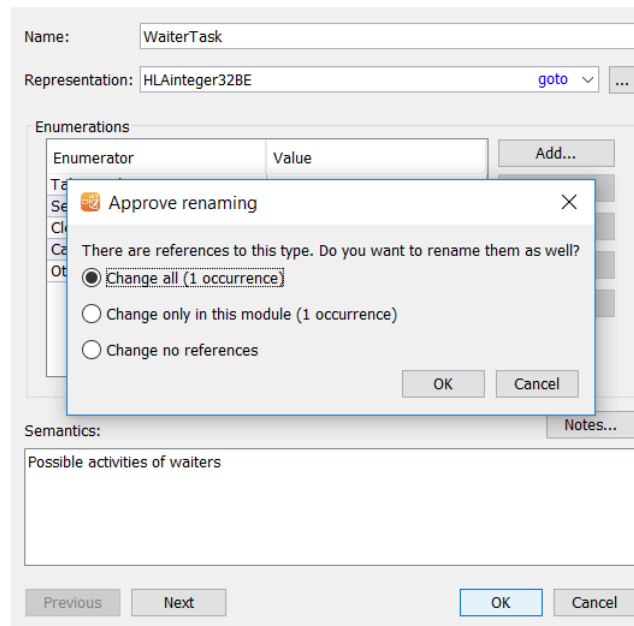


Figure 57. Approve renaming dialog.

If you change the name of an element that is referenced in your project (for example, an enumerated datatype used by an attribute), you will get an *Approve renaming* dialog, where you can select how to perform this refactoring. You can:

- **Change all** references in the project
- **Change only in this module** and do not change references in other FOM modules
- **Change no references** but only the element itself

You can also *Cancel* the operation if you do not want to change the name.

9.2. Tips for splitting a FOM into modules

When importing a FOM from earlier versions of the standard, there might be a need to split it into several modules to be able to take advantage of the flexibility of FOM modules.

The easiest way to do this is to create new modules containing the tables that you want and then set up the dependencies. Then, you can drag and drop the elements into the modules as described in [Section 6.2](#). Start with the class trees of the modules that are stand-alone, so that you can scaffold those classes (see [Section 6.6](#)) when constructing class

trees in dependent modules.

While restructuring, keep an eye on the Issue List to check that you do not create inconsistencies.

It is recommended that you move all data to new modules. If one module is to be split into three: create three new modules and move all data there, instead of creating two new and moving only some data, leaving the rest in the original module. Doing it the recommended way means you have to take a look at all parts of the FOM, and will not risk leaving some elements in the original module by mistake, or the original module becoming a storage place for assorted elements that do not fit anywhere else.

When restructuring an old FOM, look out for elements that are not used anymore and tables that are not needed.

9.3. Verifying RTI compliance

In the Tools menu, there is a tool to check RTI compliance. This tool tests the current project against the criteria an RTI uses to determine if a FOM is valid. If your project passes this test, it will load into an RTI.

9.4. Export to PDF

In the Tools menu, there is an option to generate a PDF document based on the project. This will create a PDF document with all the information in the project and FOM modules in a readable format. It is also possible to limit the scope of the document by selecting which individual modules and tables will be included.

If the "Include inherited datatypes" option is selected, datatypes used by a module will be included even if they are defined in another module which isn't being included in the document.

If you want to include the MOM classes and datatypes in the document, make sure that the "Hide MOM" option is *not* selected in the "Project Properties".

9.5. Export to RTF

Also in the Tools menu is an option to generate an RTF document based on the FOM. This is not a full export of the FOM, but rather is meant to be used as input to a Federation Agreement Document (FAD). The resulting RTF document can be edited with most word processing applications to add new columns to tables, change formatting, cut and paste into other documents, etc. We recommend using Microsoft Word to edit the document.

9.6. Importing and exporting the HLA 1516-2000 format

To import a FOM from HLA 1516-2000, open the Convert menu and select "Import HLA 1516-200 FOM...". In the shown dialog, choose a FOM to import and select a location to store the resulting FOM module. The imported module will automatically be added to the current project.

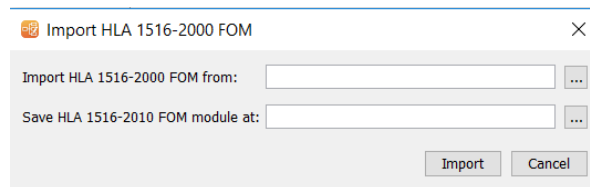


Figure 58. Import dialog.

To export your project to a HLA 1516-2000 FOM, open the Convert menu and select "Export modules to HLA 1516-2000 FOM...". In the shown dialog, choose which of the modules in your project to include in the exported FOM.

During export of a modular FOM to this format, the FOM modules will be merged. By default, the identification table of a merged FOM is empty since there is no way to merge identification data. If you want to include identification data in the export, check "Supply Identification Table" and choose the module whose identification data should be used. A tip is to have a module that contains only an identification table to use for export.

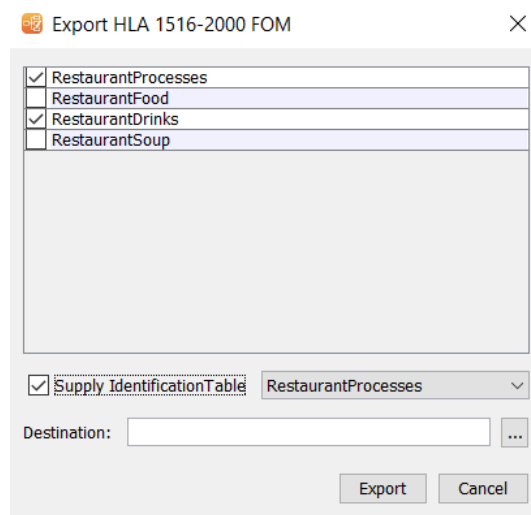


Figure 59. Export dialog.

9.7. Importing and exporting the HLA 1.3 OMT format

To export an HLA 1.3 OMT file, open the Convert menu and select "Export modules to HLA 1.3 OMT file". The export dialog looks just like the HLA 1516-2000 export dialog.

The exported OMT file might need some manual verification. The OMT file for HLA 1.3 does not contain complete definitions of things like data types, ownership and update information. Also, some elements have changed from HLA 1.3 to HLA 1516. For example, Routing Space is now Dimension and Unicode is used for names instead of ASCII.

To import an OMT file from HLA 1.3, open the Convert menu and select "Import HLA 1.3 OMT file". In the shown dialog, select an OMT file. The imported module will automatically be added to the current project.

Before importing an OMT file it might need some manual fixing. A name can consist of any combination of valid ascii characters, but it must start with a standard ascii letter.

The imported module might need some manual verification, since the OMT file does not contain complete definitions of things like datatypes, ownership and update information, and some elements, like Routing Spaces, have changed.

All automatically created datatypes will be marked with the note OMT_AUTOGEN_DATATYPE, and all created dimensions will be marked with the note VOMT_AUTOGEN_DIMENSION. They should be manually verified. Remember that you can use the Search field to search for all usages of a specific note.

9.8. Importing and exporting the HLA 1.3 FED format

To export an HLA 1.3 FED file, open the Convert menu and select "Export modules to HLA 1.3 FED file". The export dialog looks just like the HLA 1516-2000 export dialog.

To import a FED file, open the Convert menu and select "Import HLA 1.3 FED file". In the shown dialog, select a FED file. The imported module will automatically be added to the current project.

A FED file only contains parts of the FOM, and while the imported module will be accepted by an RTI, it is recommended to manually edit it to add missing information, like datatypes.

9.9. Saving a Merged HLA 1516-2010 FOM

To save your project to a merged HLA 1516-2010 FOM, open the Convert menu and select "Export Merged HLA 1516-2010 FOM...". In the shown dialog, select which of the modules in your project to include in the saved FOM.

If you want to include identification data in the merged FOM, check "Supply Identification Table" and select the module whose identification data should be used.

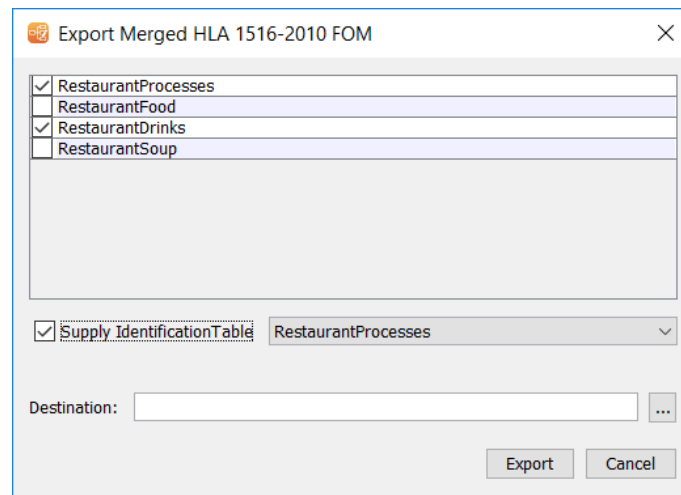


Figure 60. Export merged HLA 1516-2010 FOM dialog.

9.10. Importing and exporting the HLA 4 FOM format

To export an HLA 4 FOM file, open the Convert menu and select "Export modules to HLA 4 format...". In the shown dialog, select which of the modules in your project to export to HLA 4 format.

Dimensions in HLA 4 are located on object classes instead of attributes. This means that any dimension selected on an attribute will instead be applied to the object class during export.

To import an HLA 4 FOM file, open the Convert menu and select "Import HLA 4 FOM...". In the shown dialog, select an HLA 4 FOM file.

Features unique to HLA 4, such as directed interactions will be ignored during import. Dimensions on an object class will be applied to all attributes of that object class.

9.11. Comparing Modules

To compare one or more FOM modules from the current project with one or more other FOM modules, select "Compare modules" from the Tools menu. In the Compare Modules dialog, you can select which modules from the current project to include and which other FOM modules to compare with.

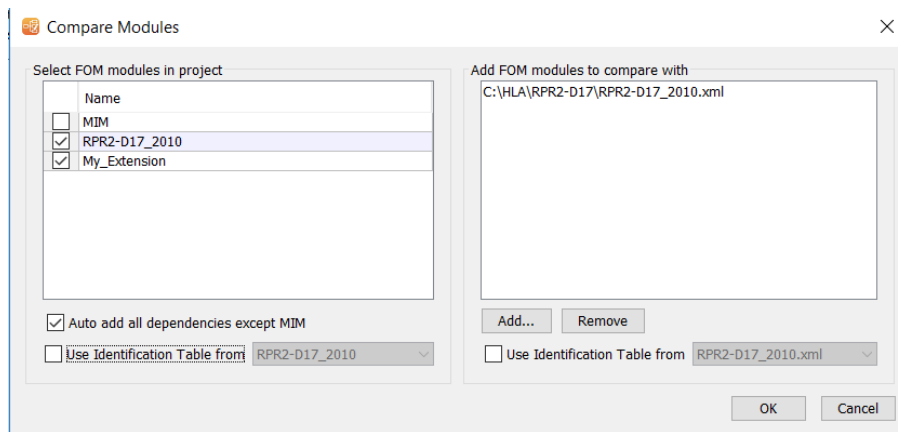


Figure 61. Compare Modules dialog.

Use the "Auto add all dependencies except MIM" checkbox to select whether the FOM modules' dependencies should automatically be added.

Use the **[Add]** and **[Remove]** buttons to add and remove FOM modules to compare with. It is also possible to select which identification table that should be used, both for the project FOM modules and for the FOM modules loaded from file.

Select OK to perform the comparison.

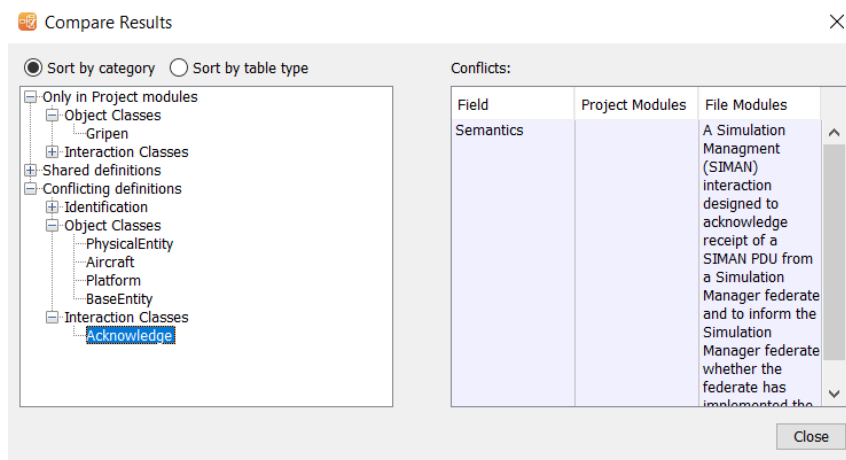


Figure 62. Compare Results dialog.

The Compare Results dialog shows the results of the comparison. The results can be sorted by category:

- **Only in Project modules** contains all definitions that only exist in the project FOM modules.
- **Only in File modules** contains all definitions that only exist in the FOM modules loaded from file.
- **Shared definitions** contain all definitions that exist and are identical in both the project and the modules loaded from file.
- **Conflicting definitions** contains all definitions that exist in the project and in the file

modules where the definitions conflict. The Conflicts table to the right contains information on the type of conflict and the definitions in the project and in the modules loaded from file.

It is also possible to sort the result by table type.

9.12. Uniform DDM Tool

The Uniform DDM tool can be used as a simple way to attach dimensions to all attributes of an Object Class. It can also be used to quickly get an overview of how dimensions are currently used in the project.

The tool follows the "Uniform DDM" definition, which has the intention of simplifying DDM in HLA. Instead of attaching a dimension to an attribute, the dimension is attached to all attributes of an Object Class, which translates to the dimension in essence being attached to the Object Class. This is considered best practice by the HLA Tutorial and is used by Pitch Developer Studio 4.0 (and later) when using DDM.

9.12.1. Uniform DDM definition

Uniform DDM is defined by three states that an Object Class can be in for each existing dimension in a project. The three states are:

- **Uniform** - The Object Class has the dimension uniformly applied, i.e. the dimension is attached to all attributes of the Object Class, including inherited attributes.
- **Not used** - None of the attributes that are not inherited have the dimension attached. However, inherited attributes may still have it attached.
- **Non-Uniform** - The Object Class has the dimension attached to at least one attribute that is not inherited, but does not fulfill the requirements for it to be considered *Uniform*. In other words, if the state of the Object Class is neither *Not Used* nor *Uniform*, then it is *Non-Uniform*.

9.12.2. Working with the Uniform DDM tool

The Uniform DDM tool can be found in the Tools menu. On the left is a list of all dimensions available in your current project. Some information about the selected dimension is displayed below. On the right is an aggregated Object Class tree. The Object Classes will be colored as explained by the legend in the bottom left, which is based on the definition in [Section 9.12.1](#).

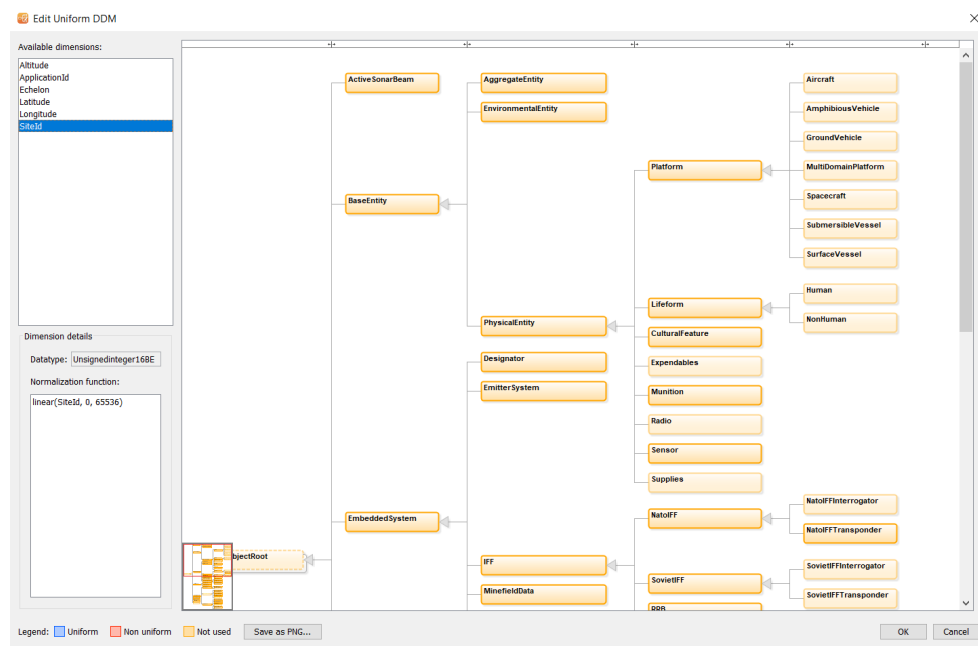


Figure 63. The Uniform DDM tool.

To attach dimensions, click on an Object Class which is not already Uniform. The class will become Uniform, and the change will propagate upwards towards the root node as well as down to all subclasses. In [Figure 64](#), Class BaseEntity was clicked on.

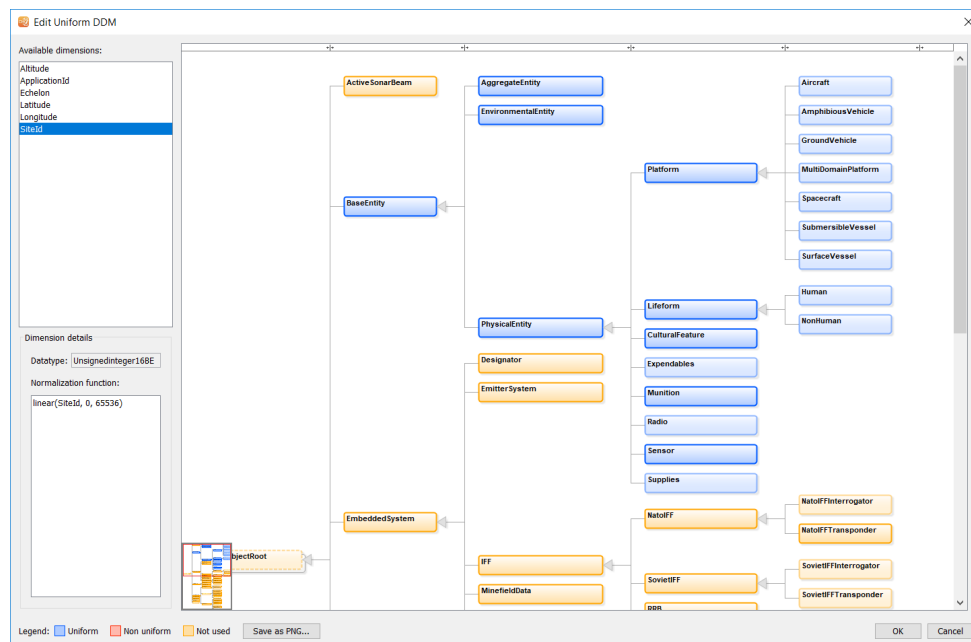


Figure 64. Applying uniform DDM to a class.

To remove dimensions, click on a Uniform Object Class. The dimension will be removed from all attributes, and the change will propagate down to all subclasses. In [Figure 65](#), Class `PhysicalEntity` was clicked on.

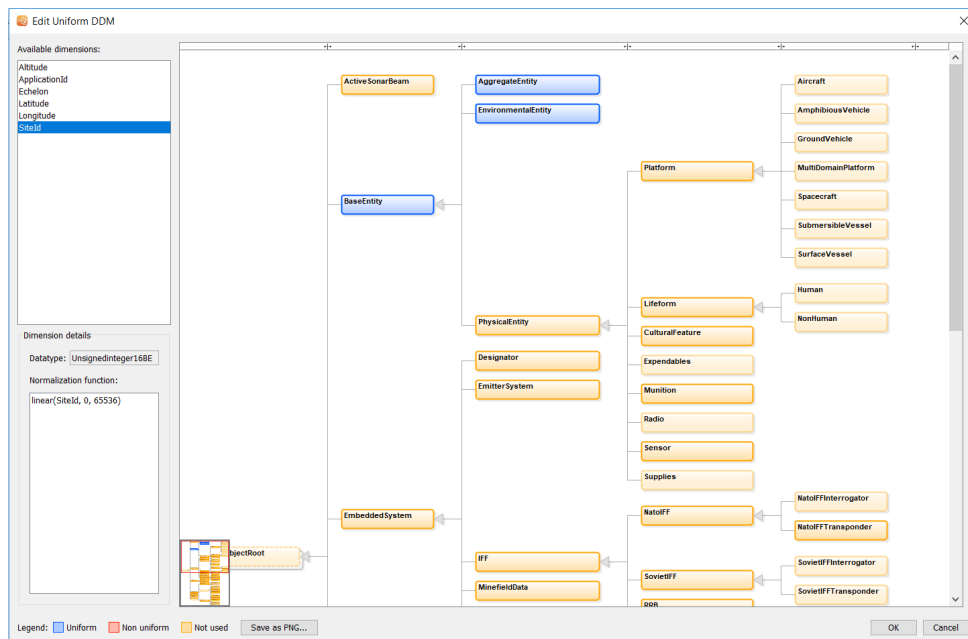


Figure 65. Removing uniform DDM from a class.

If you intend to attach a dimension to some subclasses of a class, but not all of them, then you can either:

- Attach the dimension to the parent class (which will propagate down) and then remove the dimension from each subclass in which you don't need it.
- Attach the dimension directly to each subclass in which you need it.

The result will be identical in both cases.

An Object Class that is shown as transparent is disabled and cannot be clicked on. This happens if either:

- The Object Class has at least one attribute that is defined in a module that is read-only. This includes inherited attributes.
- The Object Class has no attributes that are not inherited.

Changes made in the Uniform DDM tool are not reflected in your project until you have closed the dialog by selecting OK.

9.12.3. Project settings

If the "Use Uniform DDM" option is selected in the project settings, you will receive extra help when working with Uniform DDM. When the option is enabled, a warning will appear in the issue list whenever an Object Class is non-uniform with respect to a dimension. The recommended action is then to open the Uniform DDM Tool and attach the dimension to the Object Class. In addition, the option to attach and remove dimensions on individual attributes (in the "Edit Attribute Dialog") will be disabled. And lastly, whenever you add a new attribute to an Object Class, the new attribute will receive dimensions

according to the current Uniform DDM state of the Object Class.

9.13. HLA 4 Extended Merge

The extended merge capability in HLA 4 makes it possible, when merging modules, to:

- Add more attributes to existing object classes.
- Add more parameters to existing interaction classes.
- Add more enumerators to existing enumerated datatypes.
- Add more alternatives to existing variant records.
- Add dimensions to existing attributes and parameters.

The inspection rules have been updated to support the extended merge.

"Use HLA 4 preview features" has to be enabled for the project to use the extended merge capability, see [Section 5.3](#).

9.13.1. Example of adding attributes to an existing class

To add attributes to an existing object class, make sure that the module is dependent on the module that contains the object class to extend. Use the *Copy definition to module* function in the drop-down menu to copy the object class definition to the module, see [Figure 66](#). The object class definition will not contain any of the attributes defined in the dependant module.

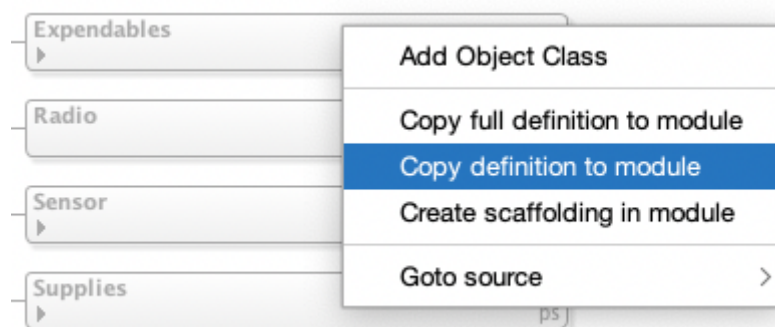


Figure 66. Copy definition to module.

New attributes can now be added to the object class. To see the attributes from definitions in other modules and all inherited attributes, use the *Show inherited Attributes* function in the drop-down menu.

An example can be seen in [Figure 67](#). On the object class Platform we can see some attributes that are defined in this module (e.g., SourceUnit), some attributes defined in dependent modules (e.g., PaintScheme), and some inherited attributes (e.g., Status).

PhysicalEntity		
Status	ActiveStatusEnum8	ps da ro
Activity	AggregateMissionEnum16	ps da ro
SymbolId	SymbolIdentifier	ps da ro

Platform		
SourceUnit	UUID	ps da ro
EmbeddedUnitList	ArrayOfUuid	ps da ro
Callsign	HLAUnicodeString	ps da ro
Route	PathVariantStruct	ps da ro
Destination	PointVariantStruct	ps da ro
PaintScheme	AppearancePaintSc...	ps da ro
MobilityKilled	RPRboolean	ps da ro
Damage	AppearanceDamage...	ps da ro
IsSmokeEmanating	RPRboolean	ps da ro
IsEngineEmittingSm...	RPRboolean	ps da ro
IsFlaming	RPRboolean	ps da ro
PowerPlantOn	RPRboolean	ps da ro
AmmunitionSupplyC...	RPRboolean	ps da ro
FuelSupplyCapability	RPRboolean	ps da ro
RecoveryCapability	RPRboolean	ps da ro
RepairCapability	RPRboolean	ps da ro
Status	ActiveStatusEnum8	ps da ro
Activity	AggregateMissionE...	ps da ro
SymbolId	SymbolIdentifier	ps da ro

Figure 67. Attributes defined in other modules and inherited attributes are displayed in blue.

The *Copy definition to module* function is also available on interaction classes, so the same approach can be used to add parameters to existing interaction classes.

9.13.2. Example of extending enumerations and variant records

Use drag and drop to drag the datatype from the enumerations or variant records table in the original module and drop it in the corresponding table in the new module (as described in [Section 7](#)). To extend the enumeration, double-click on the enumerated datatype and add a new enumerator, see [Figure 68](#).

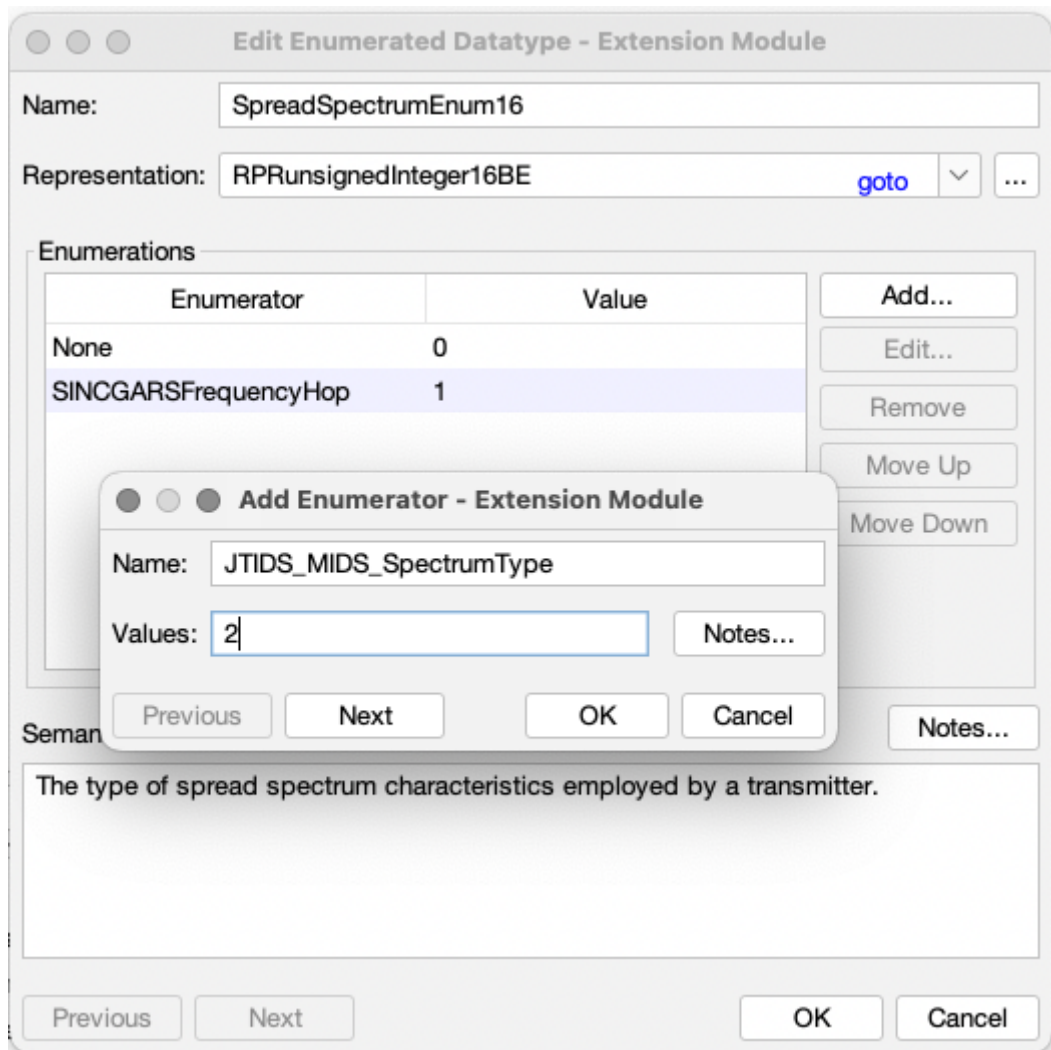


Figure 68. An extended enumeration.

The extended enumerator can now be used to extend the variant record. Double-click on the variant record datatype and add a new alternative, see [Figure 69](#). The encoding of the extended variant record should be `HLAextendableVariantRecord`, otherwise a warning will be generated.

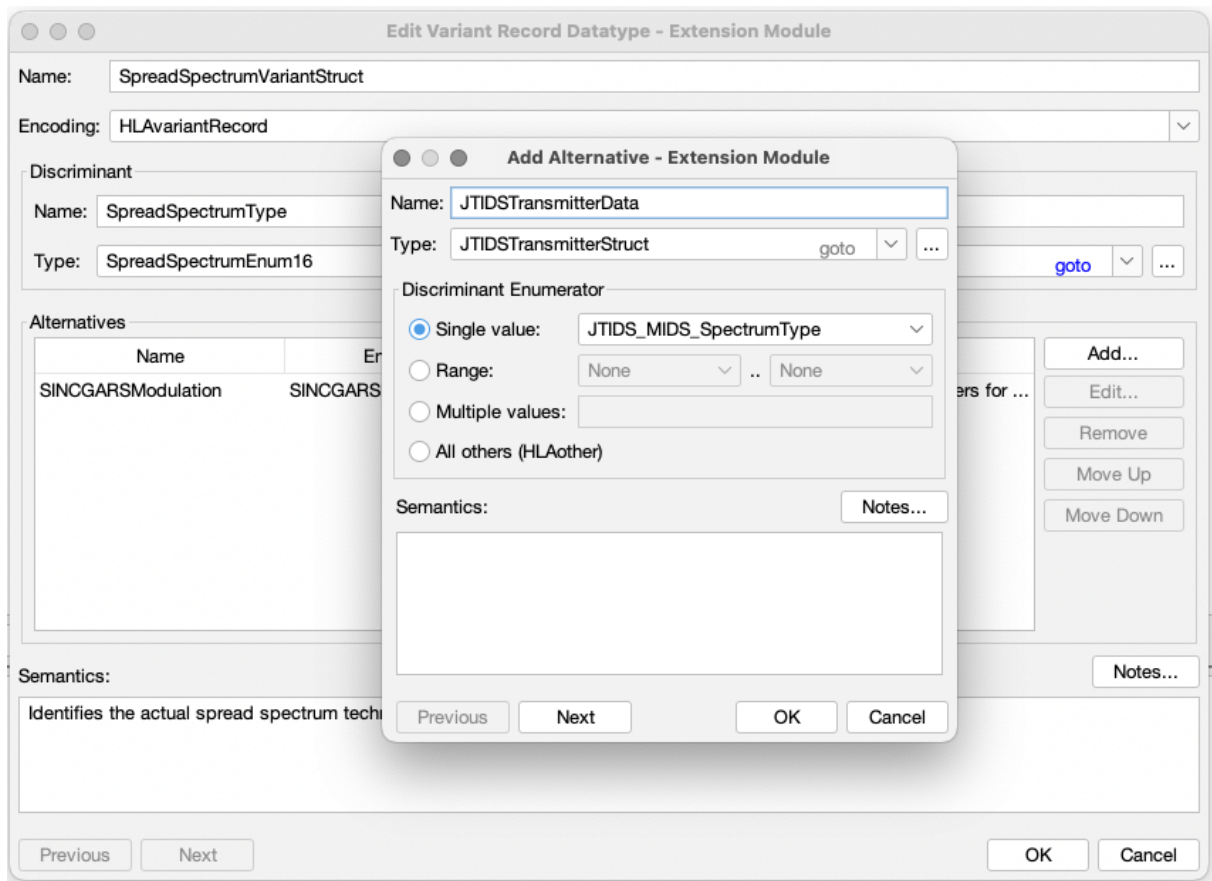


Figure 69. An extended variant record.

For more information on extended merging, see www.pitchtechnologies.com/2019/01/new-object-modeling-opportunities-in-hla-4.



Pitch Technologies

Repslagaregatan 25, S-582 22 Linköping, Sweden

Tel: +46 (0)13 470 55 00

Web: <https://www.pitchtechnologies.com>, Email: info@pitchtechnologies.com